SKILLSWAP · STUDENT SKILL SHARING · FASTAPI BACKEND

The 16 Technical Requirements

How we implemented each one — code, endpoints, inputs and outputs

This document explains every mandatory technical requirement in detail, in groups of three, so that each team member can present one group. For each requirement you will find what it means, how we used it in the SkillSwap code, the exact code, the endpoints it affects, and **real requests and responses** recorded from our running API with PostgreSQL.

Group	Presenter	Requirements
1	Member 1: _____	1 FastAPI REST API · 2 CRUD APIs · 3 PostgreSQL Database
2	Member 2: _____	4 Database Integration · 5 Pydantic Validation · 6 JWT Authentication
3	Member 3: _____	7 Password Hashing · 8 Authorization · 9 RBAC
4	Member 4: _____	10 Protected Routes · 11 Path & Query Parameters · 12 HTTP Status Codes
5	Member 5: _____	13 Exception Handling · 14 Environment Variables · 15 Swagger / OpenAPI · 16 Postman Testing

Group 5 has four requirements because 16 does not divide into groups of three; requirement 16 (Postman) is short and is shown live at the end.

Before we start: the project in one page

SkillSwap is the backend (REST API) of a student skill-sharing platform. Students who can teach become **mentors** and publish **skill profiles**. Other **students** search for skills or mentors, **book** a mentoring session, can **cancel** it, see their **session history** and leave a **review and rating**. An **admin** manages users and watches statistics.

Role	Who	Main actions
student	learns a skill	search, book, cancel, view history, review
mentor	teaches a skill	create/edit/delete own skills, complete or cancel own sessions
admin	manages the platform (created from .env)	list/search users, change role, activate/deactivate/delete users, statistics

Entity (table)	What it stores
User (users)	every account with role and hashed password
Skill (skills)	a skill profile: title, category, level, description, owner mentor
MentoringSession (sessions)	a booking: student, mentor, skill, time, duration, status booked / completed / cancelled
Review (reviews)	rating 1–5 and comment for one completed session

The people in our examples

All examples in this document are **real** requests sent to our running API and the real answers it gave. We used two demo accounts: **Ananya Rao**, a mentor (user id 9) who teaches “HTML & CSS Basics” (skill id 7), and **Kiran Kumar**, a student (user id 10) who books session 7 with her, reviews it, and books and cancels session 8. The admin is **admin@skillswap.com**.

How to read the examples

Example: the health-check endpoint

REQUEST `GET /` (no token)

RESPONSE 200 OK

```
{
  "status": "ok",
  "docs": "/docs"
}
```

- **REQUEST** (blue): the HTTP method, the URL path, whether a login token was sent, and the JSON body if there is one.
- **RESPONSE**: the status code and the JSON the API sent back. **Green** = success, **red** = an error we return on purpose.
- Times are returned in Indian time (+05:30) because that is the database time zone; we send them in UTC (Z). Both mean the same moment.
- Long lists and tokens are shortened with “…”. Everything else is exactly what the API returned.

All 29 endpoints at a glance

Method	Path	Access	Success	Possible error codes
POST	/auth/register	public	201	409, 422
POST	/auth/login	public	200	401, 403, 422

Method	Path	Access	Success	Possible error codes
GET	/users/me	logged in	200	401
PUT	/users/me	logged in	200	401, 422
GET	/users/mentors	public	200	422
GET	/users/mentors/{mentor_id}	public	200	404, 422
POST	/skills	mentor	201	401, 403, 409, 422
GET	/skills	public	200	422
GET	/skills/{skill_id}	public	200	404, 422
PUT	/skills/{skill_id}	mentor, admin	200	401, 403, 404, 409, 422
DELETE	/skills/{skill_id}	mentor, admin	204	400, 401, 403, 404, 422
POST	/sessions	student	201	400, 401, 403, 404, 409, 422
GET	/sessions	logged in	200	401, 422
GET	/sessions/{session_id}	logged in	200	401, 403, 404, 422
PATCH	/sessions/{session_id}/cancel	logged in	200	400, 401, 403, 404, 422
PATCH	/sessions/{session_id}/complete	mentor, admin	200	400, 401, 403, 404, 422
POST	/reviews	student	201	400, 401, 403, 404, 409, 422
GET	/reviews	public	200	422
GET	/reviews/{review_id}	public	200	404, 422
PUT	/reviews/{review_id}	logged in	200	401, 403, 404, 422
DELETE	/reviews/{review_id}	logged in	204	401, 403, 404, 422
GET	/admin/users	admin	200	401, 403, 422
GET	/admin/users/{user_id}	admin	200	401, 403, 404, 422
PATCH	/admin/users/{user_id}/role	admin	200	400, 401, 403, 404, 422
PATCH	/admin/users/{user_id}/status	admin	200	400, 401, 403, 404, 422
DELETE	/admin/users/{user_id}	admin	204	400, 401, 403, 404, 422
DELETE	/admin/sessions/{session_id}	admin	204	401, 403, 404, 422
GET	/admin/stats	admin	200	401, 403
GET	/	public	200	—

This table was produced by reading our source code automatically, so it matches the code exactly.

GROUP 1 · PRESENTED BY MEMBER 1 (name: _____)

Building the API and storing data

Requirement 1 · FastAPI REST API
Requirement 2 · CRUD APIs
Requirement 3 · PostgreSQL Database

This group explains the foundation: how the web API is built with FastAPI, how every main entity supports Create, Read, Update and Delete, and where all the data is stored.

REQUIREMENT 1

FastAPI REST API

Group 1
Member 1

> **Expected implementation:** *Build the backend using FastAPI with proper API routes.*

1.1 What it means

FastAPI is a Python framework for building web APIs. A **REST API** is a set of URLs (endpoints) where each URL is a **resource** (a noun like `/skills`) and the **HTTP method** is the action: `POST` = create, `GET` = read, `PUT`/`PATCH` = update, `DELETE` = delete. Data goes in and out as JSON.

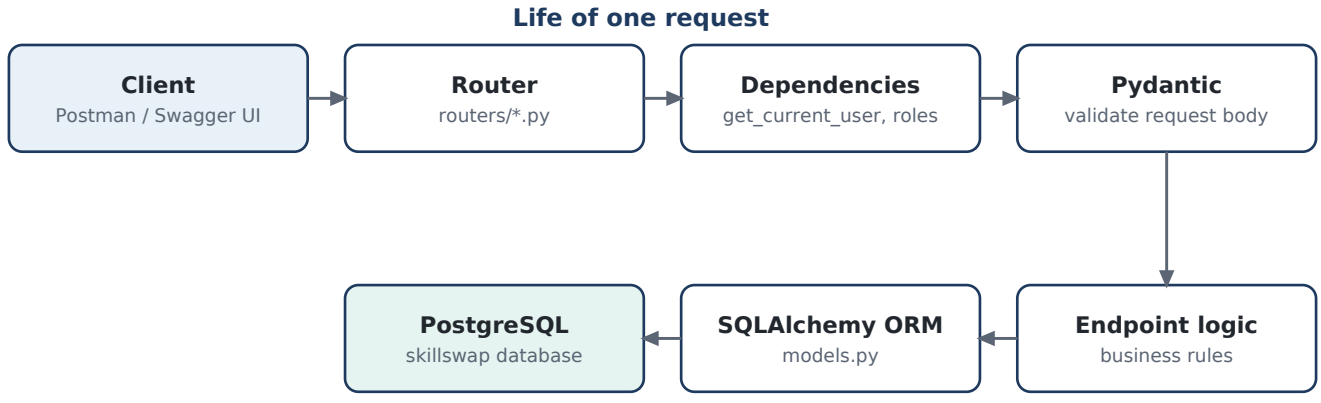
1.2 How we used it in SkillSwap

- One FastAPI application object in `app/main.py`, served by Uvicorn (`uvicorn app.main:app --reload`).
- Routes are organised in **6 routers** (`APIRouter`), one per topic. Each router has a URL prefix and a tag (the group name in Swagger).
- **29 endpoints** in total (table on page 2). URLs are nouns, methods are verbs, ids go in the path.
- Every endpoint declares the shape of its answer with `response_model` and its success code with `status_code`.
- On startup a `lifespan` function creates the database tables and the admin account.

1.3 Where in the code

File	Role in the API
app/main.py	creates the FastAPI app, startup tasks, global error handler, includes all routers
app/routers/auth.py	/auth — register, login
app/routers/users.py	/users — my profile, mentor search, mentor profile
app/routers/skills.py	/skills — skill CRUD and search
app/routers/sessions.py	/sessions — book, history, get, cancel, complete
app/routers/reviews.py	/reviews — review CRUD
app/routers/admin.py	/admin — user management, delete session, statistics

1.4 The code



The JSON response (e.g. 201 Created) travels back the same way to the client

Figure. How a request moves through our files.

```
app/main.py

@asynccontextmanager
async def lifespan(app: FastAPI):
    Base.metadata.create_all(bind=engine)
    create_default_admin()
    yield


app = FastAPI(
    title="SkillSwap API",
    description="Student skill sharing platform: skill profiles, mentor search, session booking and reviews.",
    version="1.0.0",
    lifespan=lifespan,
)

app.include_router(auth.router)
app.include_router(users.router)
app.include_router(skills.router)
app.include_router(sessions.router)
app.include_router(reviews.router)
app.include_router(admin.router)


@app.get("/", tags=["Health"])
def health():
    return {"status": "ok", "docs": "/docs"}
```

Code	What it means
`async def lifespan(app)`	Runs once when the server starts: creates missing tables and the default admin, then the app serves requests.
`app = FastAPI(title=..., ...)`	The application object. Title, description and version appear in Swagger.
`app.include_router(skills.router)`	Plugs all /skills endpoints into the app. Same for the other 5 routers.
`@app.get("/", tags=["Health"])`	A simple endpoint to check the server is alive.

```
app/routers/skills.py — a router and one route

router = APIRouter(prefix="/skills", tags=["Skills"])


@router.get("/{skill_id}", response_model=SkillOut)
def get_skill(skill_id: int, db: Session = Depends(get_db)):
    return get_skill_or_404(skill_id, db)
```

Code	What it means
`APIRouter(prefix="/skills", tags=["Skills"])`	All routes in this file start with /skills and are grouped under “Skills” in Swagger.
`@router.get("/{skill_id}", response_model=SkillOut)`	GET /skills/{skill_id}. The answer is converted to the SkillOut shape.
`skill_id: int`	A path parameter. FastAPI converts it to an int; “abc” gives 422.
`db: Session = Depends(get_db)`	FastAPI gives the function a database session (requirement 4).

1.5 Input → output

Reading one skill

REQUEST `GET /skills/7` (no token)

RESPONSE 200 OK


```
{
  "id": 7,
  "mentor_id": 9,
  "title": "HTML & CSS Basics",
  "category": "Web Development",
  "level": "beginner",
  "description": "Build your first web page with HTML and style it with CSS",
  "created_at": "2026-10-02T16:54:16.802468+05:30"
}
```


FastAPI checks the path type automatically

REQUEST `GET /skills/abc` (no token)

RESPONSE 422 Unprocessable Content


```
{
  "detail": [
    {
      "type": "int_parsing",
      "loc": [
        "path",
        "skill_id"
      ],
      "msg": "Input should be a valid integer, unable to parse string as an integer"
    }
  ]
}
```


Say this when presenting

- “Our backend is one FastAPI app with 6 routers and 29 REST endpoints.”
- “The URL is the resource, the HTTP method is the action, data is JSON.”
- Show main.py (include_router) and one router file, then the endpoint table on page 2.

REQUIREMENT 2

CRUD APIs

Group 1
Member 1

> **Expected implementation:** *Implement Create, Read, Update and Delete operations for the main entities.*

2.1 What it means

CRUD = **C**reate, **R**ead, **U**pdate, **D**elete — the four basic things you do with stored data. In REST they map to POST, GET, PUT/PATCH and DELETE.

2.2 How we used it in SkillSwap

Entity	Create	Read	Update	Delete
User	POST /auth/register	GET /users/me GET /admin/users GET /admin/users/{id}	PUT /users/me PATCH /admin/users/{id}/role PATCH /admin/users/{id}/status	DELETE /admin/users/{id}
Skill	POST /skills	GET /skills GET /skills/{id}	PUT /skills/{id}	DELETE /skills/{id}
Session	POST /sessions	GET /sessions GET /sessions/{id}	PATCH /sessions/{id}/cancel PATCH /sessions/{id}/complete	DELETE /admin/sessions/{id}
Review	POST /reviews	GET /reviews GET /reviews/{id}	PUT /reviews/{id}	DELETE /reviews/{id}

- Create → **201 Created** and the new object. Read / Update → **200 OK**. Delete → **204 No Content** (empty body).
- Updates are **partial**: only the fields you send are changed (`model_dump(exclude_unset=True)`).
- Sessions are updated by changing their **status** (cancel / complete) so the history is kept.
- Each operation also has rules: only the owner may update/delete, duplicates are refused, and so on (requirements 8 and 13).

2.3 The code — full CRUD for skills

```
Create

@router.post("", response_model=SkillOut, status_code=status.HTTP_201_CREATED)
def create_skill(
    data: SkillCreate,
    user: User = Depends(require_roles(Role.MENTOR)),
    db: Session = Depends(get_db),
):
    check_duplicate_title(db, user.id, data.title)
    skill = Skill(mentor_id=user.id, **data.model_dump())
    db.add(skill)
    db.commit()
    db.refresh(skill)
    return skill
```

Code	What it means
`@router.post("", ..., status_code=...201_CREATED)`	POST /skills answers 201 Created.
`data: SkillCreate`	The JSON body, already validated by Pydantic (requirement 5).
`user = Depends(require_roles(Role.MENTOR))`	Only mentors can create (requirement 9).
`check_duplicate_title(...)`	409 if the mentor already has a skill with this title.

Code	What it means
`Skill(mentor_id=user.id, **data.model_dump())`	Build the row; the owner comes from the token, not from the body.
`db.add(skill); db.commit(); db.refresh(skill)`	INSERT into PostgreSQL, save, and reload to get the new id and created_at.

Update

```
@router.put("/{skill_id}", response_model=SkillOut)
def update_skill(
    skill_id: int,
    data: SkillUpdate,
    user: User = Depends(require_roles(Role.MENTOR, Role.ADMIN)),
    db: Session = Depends(get_db),
):
    skill = get_skill_or_404(skill_id, db)
    check_owner(skill, user)
    changes = data.model_dump(exclude_unset=True)
    if changes.get("title"):
        check_duplicate_title(db, skill.mentor_id, changes["title"], exclude_id=skill.id)
    for field, value in changes.items():
        setattr(skill, field, value)
    db.commit()
    db.refresh(skill)
    return skill
```

Code	What it means
`get_skill_or_404(skill_id, db)`	Load the row or answer 404.
`check_owner(skill, user)`	Only the owner or an admin may update (403 otherwise).
`data.model_dump(exclude_unset=True)`	Only the fields that were sent, e.g. just “level”.
`setattr(skill, field, value); db.commit()`	Change those columns and save: an UPDATE statement.

Delete

```
@router.delete("/{skill_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_skill(
    skill_id: int,
    user: User = Depends(require_roles(Role.MENTOR, Role.ADMIN)),
    db: Session = Depends(get_db),
):
    skill = get_skill_or_404(skill_id, db)
    check_owner(skill, user)
    # Business rule: cannot delete a skill that still has upcoming booked sessions
    upcoming = db.scalar(
        select(MentoringSession).where(
            MentoringSession.skill_id == skill.id,
            MentoringSession.status == SessionStatus.BOOKED,
            MentoringSession.scheduled_at > datetime.now(timezone.utc),
        )
    )
    if upcoming:
        raise HTTPException(status.HTTP_400_BAD_REQUEST, "Cancel upcoming sessions for this skill first")
    db.delete(skill)
    db.commit()
    return Response(status_code=status.HTTP_204_NO_CONTENT)
```

Code	What it means
`upcoming = db.scalar(select(MentoringSession)...)`	Business rule: is there a future booked session for this skill?
`raise HTTPException(...400...)`	If yes, refuse to delete.
`db.delete(skill); db.commit()`	DELETE the row.
`Response(status_code=204)`	Success with an empty body.

Read (list + search) is shown in requirement 11, and read-one in requirement 1.

2.4 Input → output: the full CRUD cycle

C — Create a skill

REQUEST `POST /skills` (Authorization: Bearer <token>)

```
{
  "title": "HTML & CSS Basics",
  "category": "Web Development",
  "level": "beginner",
  "description": "Build your first web page with HTML and style it with CSS"
}
```

RESPONSE 201 Created

```
{
  "id": 7,
  "mentor_id": 9,
  "title": "HTML & CSS Basics",
  "category": "Web Development",
  "level": "beginner",
  "description": "Build your first web page with HTML and style it with CSS",
  "created_at": "2026-10-02T16:54:16.802468+05:30"
}
```

R — Read: list a mentor's skills (paged, limit 2)

REQUEST `GET /skills?mentor_id=9&skip=0&limit=2` (no token)

RESPONSE 200 OK

```
[
  {
    "id": 7,
    "mentor_id": 9,
    "title": "HTML & CSS Basics",
    "category": "Web Development",
    "level": "beginner",
    "description": "Build your first web page with HTML and style it with CSS",
    "created_at": "2026-10-02T16:54:16.802468+05:30"
  },
  {
    "id": 8,
    "mentor_id": 9,
    "title": "JavaScript Fundamentals",
    "category": "Web Development",
    "level": "intermediate",
    "description": "Variables, functions, DOM and events",
    "created_at": "2026-10-02T16:54:16.808249+05:30"
  }
]
```

U — Update: change only level and description

REQUEST `PUT /skills/7` (Authorization: Bearer <token>)

```
{
  "level": "intermediate",
  "description": "HTML5 page structure, CSS flexbox and grid"
}
```

RESPONSE 200 OK

```
{
  "id": 7,
  "mentor_id": 9,
  "title": "HTML & CSS Basics",
  "category": "Web Development",
  "level": "intermediate",
  "description": "HTML5 page structure, CSS flexbox and grid",
  "created_at": "2026-10-02T16:54:16.802468+05:30"
}
```

D — Delete skill 9 (Git for Beginners)

REQUEST `DELETE /skills/9` (Authorization: Bearer <token>)

RESPONSE 204 No Content

```
(empty body)
```

Deleting it again: it is gone

REQUEST `DELETE /skills/9` (Authorization: Bearer <token>)

RESPONSE 404 Not Found

```
{
  "detail": "Skill not found"
}
```

CRUD on users: update my profile

REQUEST `PUT /users/me` (Authorization: Bearer <token>)

```
{
  "bio": "MCA student, teaches HTML, CSS and JavaScript"
}
```

RESPONSE 200 OK

```
{
  "id": 9,
  "name": "Ananya Rao",
  "email": "ananya.mentor@skillswap.com",
  "role": "mentor",
  "bio": "MCA student, teaches HTML, CSS and JavaScript",
  "is_active": true,
  "created_at": "2026-10-02T16:54:15.633358+05:30"
}
```

Say this when presenting

- “All four main entities — users, skills, sessions, reviews — have Create, Read, Update and Delete.”
- Walk through the CRUD table, then show create/update/delete for skills with the real outputs.
- “Create gives 201, delete gives 204 with an empty body, and updates are partial.”

REQUIREMENT 3

PostgreSQL Database

Group 1
Member 1

> **Expected implementation:** *Store application data in PostgreSQL. Supabase PostgreSQL may be used.*

3.1 What it means

PostgreSQL is a **relational database**: data is stored in tables with columns, and tables are linked with **foreign keys**. Supabase is a cloud service that runs PostgreSQL for you.

3.2 How we used it in SkillSwap

- Database **skillswap** on **PostgreSQL 18** on our laptop, viewed and managed with **pgAdmin 4**.
- Python connects with the **psycopg 3** driver. The address is one line in `.env`:

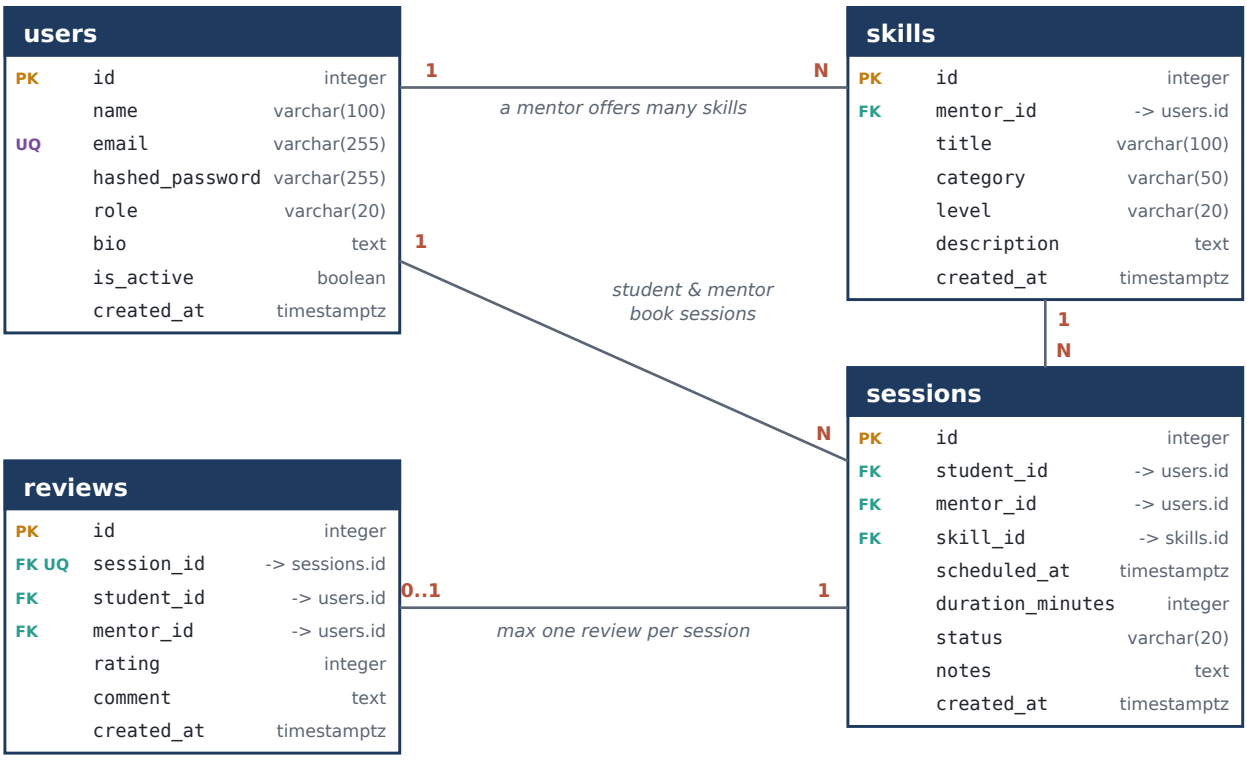
```
.env

DATABASE_URL=postgresql+psycopg://postgres:postgres@localhost:5432/skillswap
#            └ driver ──────────┘ └ user ┘ └ password┘ └ host ─┘ └port┘ └ database ┘

# Supabase: same code, only this line changes
DATABASE_URL=postgresql+psycopg://postgres:<password>@db.<project>.supabase.co:5432/postgres
```

- **4 tables**: users, skills, sessions, reviews — created automatically on startup.
- PostgreSQL enforces data rules for us: **primary keys**, **unique** email, **unique** (mentor, title), **unique** review per session, **foreign keys** with **ON DELETE CASCADE**, NOT NULL columns and time-zone aware dates.

3.3 The tables and how they are linked



PK = primary key FK = foreign key (ON DELETE CASCADE) UQ = unique

Figure. Entity–relationship diagram of the skillswap database.

3.4 Real table definitions from PostgreSQL

This is the output of `\d` in psql for each table. It proves the data is really stored in PostgreSQL with these constraints:

```
\d users

Table "public.users"
     Column      |           Type           | Collation | Nullable |              Default
-----------------+--------------------------+-----------+----------+-----------------------------------
 id              | integer                  |           | not null | nextval('users_id_seq'::regclass)
 name            | character varying(100)   |           | not null |
 email           | character varying(255)   |           | not null |
 hashed_password | character varying(255)   |           | not null |
 role            | character varying(20)    |           | not null |
 bio             | text                     |           |          |
 is_active       | boolean                  |           | not null |
 created_at      | timestamp with time zone |           | not null | now()
Indexes:
    "users_pkey" PRIMARY KEY, btree (id)
    "ix_users_email" UNIQUE, btree (email)
Referenced by:
    TABLE "reviews" CONSTRAINT "reviews_mentor_id_fkey" FOREIGN KEY (mentor_id) REFERENCES users(id) ON DELETE
    CASCADE
    TABLE "reviews" CONSTRAINT "reviews_student_id_fkey" FOREIGN KEY (student_id) REFERENCES users(id) ON
    DELETE CASCADE
    TABLE "sessions" CONSTRAINT "sessions_mentor_id_fkey" FOREIGN KEY (mentor_id) REFERENCES users(id) ON
    DELETE CASCADE
    TABLE "sessions" CONSTRAINT "sessions_student_id_fkey" FOREIGN KEY (student_id) REFERENCES users(id) ON
    DELETE CASCADE
    TABLE "skills" CONSTRAINT "skills_mentor_id_fkey" FOREIGN KEY (mentor_id) REFERENCES users(id) ON DELETE
    CASCADE
```

```
\d skills

                                    Table "public.skills"
   Column    |           Type           | Collation | Nullable |              Default
-------------+--------------------------+-----------+----------+-----------------------------------
 id          | integer                  |           | not null | nextval('skills_id_seq'::regclass)
 mentor_id   | integer                  |           | not null |
 title       | character varying(100)   |           | not null |
 category    | character varying(50)    |           | not null |
 level       | character varying(20)    |           | not null |
 description | text                     |           |          |
 created_at  | timestamp with time zone |           | not null | now()
Indexes:
    "skills_pkey" PRIMARY KEY, btree (id)
    "skills_mentor_id_title_key" UNIQUE CONSTRAINT, btree (mentor_id, title)
Foreign-key constraints:
    "skills_mentor_id_fkey" FOREIGN KEY (mentor_id) REFERENCES users(id) ON DELETE CASCADE
Referenced by:
    TABLE "sessions" CONSTRAINT "sessions_skill_id_fkey" FOREIGN KEY (skill_id) REFERENCES skills(id) ON DELETE
    CASCADE
```

```
\d sessions

                                        Table "public.sessions"
      Column      |           Type           | Collation | Nullable |               Default
------------------+--------------------------+-----------+----------+--------------------------------------
 id               | integer                  |           | not null | nextval('sessions_id_seq'::regclass)
 student_id       | integer                  |           | not null |
 mentor_id        | integer                  |           | not null |
 skill_id         | integer                  |           | not null |
 scheduled_at     | timestamp with time zone |           | not null |
 duration_minutes | integer                  |           | not null |
 status           | character varying(20)    |           | not null |
 notes            | text                     |           |          |
 created_at       | timestamp with time zone |           | not null | now()
Indexes:
    "sessions_pkey" PRIMARY KEY, btree (id)
Foreign-key constraints:
    "sessions_mentor_id_fkey" FOREIGN KEY (mentor_id) REFERENCES users(id) ON DELETE CASCADE
    "sessions_skill_id_fkey" FOREIGN KEY (skill_id) REFERENCES skills(id) ON DELETE CASCADE
    "sessions_student_id_fkey" FOREIGN KEY (student_id) REFERENCES users(id) ON DELETE CASCADE
Referenced by:
    TABLE "reviews" CONSTRAINT "reviews_session_id_fkey" FOREIGN KEY (session_id) REFERENCES sessions(id) ON
    DELETE CASCADE
```

```
\d reviews

                                     Table "public.reviews"
   Column   |           Type           | Collation | Nullable |               Default
------------+--------------------------+-----------+----------+-------------------------------------
 id         | integer                  |           | not null | nextval('reviews_id_seq'::regclass)
 session_id | integer                  |           | not null |
 student_id | integer                  |           | not null |
 mentor_id  | integer                  |           | not null |
 rating     | integer                  |           | not null |
 comment    | text                     |           |          |
 created_at | timestamp with time zone |           | not null | now()
Indexes:
    "reviews_pkey" PRIMARY KEY, btree (id)
    "reviews_session_id_key" UNIQUE CONSTRAINT, btree (session_id)
Foreign-key constraints:
    "reviews_mentor_id_fkey" FOREIGN KEY (mentor_id) REFERENCES users(id) ON DELETE CASCADE
    "reviews_session_id_fkey" FOREIGN KEY (session_id) REFERENCES sessions(id) ON DELETE CASCADE
    "reviews_student_id_fkey" FOREIGN KEY (student_id) REFERENCES users(id) ON DELETE CASCADE
```

From \d output	What it means
`nextval('users_id_seq')`	PostgreSQL creates the id numbers automatically (1, 2, 3 ...).
`"ix_users_email" UNIQUE`	Two accounts can never have the same email.
`FOREIGN KEY ... ON DELETE CASCADE`	A link to another table; deleting the user also deletes their rows in other tables.
`timestamp with time zone ... now()`	Creation time is filled in by PostgreSQL.
`UNIQUE (mentor_id, title)`	A mentor cannot list the same skill twice.
`reviews_session_id_key UNIQUE`	One review per session.

3.5 Seeing the data

```
psql / pgAdmin Query Tool — real output

skillswap=# SELECT id, name, email, role FROM users WHERE id IN (1, 9, 10) ORDER BY id;
 id |    name     |            email            |  role
----+-------------+-----------------------------+---------
  1 | Admin       | admin@skillswap.com         | admin
  9 | Ananya Rao  | ananya.mentor@skillswap.com | mentor
 10 | Kiran Kumar | kiran.student@skillswap.com | student
(3 rows)
```

In pgAdmin: Servers → skillswap → Schemas → public → Tables → right-click a table → View/Edit Data → All Rows.

Say this when presenting

- “All our data lives in a PostgreSQL 18 database called skillswap. Switching to Supabase is one line in .env.”
- Show the ER diagram and explain the 4 tables and their links.
- Show the \d output: unique email, foreign keys with cascade, automatic ids — the database itself protects the data.
- Open pgAdmin and show the rows created by the demo.

GROUP 2 · PRESENTED BY MEMBER 2 (name: _____)

Talking to the database, checking data, logging in

Requirement 4 · Database Integration
Requirement 5 · Pydantic Validation
Requirement 6 · JWT Authentication

This group explains how FastAPI reads and writes PostgreSQL through an ORM, how every piece of incoming data is checked before it is used, and how users register, log in and receive a JWT token.

REQUIREMENT 4

Database Integration

Group 2
Member 2

> **Expected implementation:** *Connect FastAPI to PostgreSQL and implement database operations using an appropriate ORM/database layer.*

4.1 What it means

An **ORM** (Object-Relational Mapper) lets us work with database rows as Python objects. Instead of writing SQL strings, we write Python and the ORM produces safe SQL.

4.2 How we used it in SkillSwap

- **SQLAlchemy 2** is our ORM. Each table is a Python class in `models.py`.
- `database.py` creates the **engine** (a pool of connections to PostgreSQL) and the `get_db` dependency.
- Every endpoint that needs the database asks for `db: Session = Depends(get_db)`: one session per request, always closed afterwards.
- Tables are created from the classes on startup (`Base.metadata.create_all`).
- Because the ORM always sends values as parameters, **SQL injection is not possible**.

4.3 The connection layer

app/database.py

```
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker

from app.config import settings

engine = create_engine(settings.database_url, pool_pre_ping=True)
SessionLocal = sessionmaker(bind=engine, autoflush=False)


class Base(DeclarativeBase):
    pass


def get_db():
    """FastAPI dependency: one DB session per request."""
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Code	What it means
`create_engine(settings.database_url, pool_pre_ping=True)`	Connect to PostgreSQL with the URL from .env. pool_pre_ping checks a connection is alive before using it.
`sessionmaker(bind=engine, autoflush=False)`	A factory for database sessions.
`class Base(DeclarativeBase)`	The parent class for all our table classes.
`def get_db(): ... yield db ... finally: db.close()`	A FastAPI dependency: open a session, give it to the endpoint, always close it — even after an error.

4.4 The models (tables as classes)

app/models.py

```
class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
    email: Mapped[str] = mapped_column(String(255), unique=True, index=True)
    hashed_password: Mapped[str] = mapped_column(String(255))
    role: Mapped[str] = mapped_column(String(20), default=Role.STUDENT)
    bio: Mapped[str | None] = mapped_column(Text)
    is_active: Mapped[bool] = mapped_column(default=True)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now())


class MentoringSession(Base):
    __tablename__ = "sessions"

    id: Mapped[int] = mapped_column(primary_key=True)
    student_id: Mapped[int] = mapped_column(ForeignKey("users.id", ondelete="CASCADE"))
    mentor_id: Mapped[int] = mapped_column(ForeignKey("users.id", ondelete="CASCADE"))
    skill_id: Mapped[int] = mapped_column(ForeignKey("skills.id", ondelete="CASCADE"))
    scheduled_at: Mapped[datetime] = mapped_column(DateTime(timezone=True))
    duration_minutes: Mapped[int]
    status: Mapped[str] = mapped_column(String(20), default=SessionStatus.BOOKED)
    notes: Mapped[str | None] = mapped_column(Text)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now())
```

Code	What it means
`Mapped[int] = mapped_column(primary_key=True)`	Integer primary key; PostgreSQL numbers the rows.
`String(255), unique=True, index=True`	Text up to 255 characters, unique, with an index for fast lookups by email.
`Mapped[str | None]`	The column may be empty (NULL).
`ForeignKey("users.id", ondelete="CASCADE")`	Must point to an existing user; deleted together with that user.
`DateTime(timezone=True)`	Stored as timestamptz, so times are exact in every time zone.
`default=SessionStatus.BOOKED`	A new session starts as booked.

4.5 Every kind of database operation we use, and the SQL it becomes

Operation	Our ORM code	SQL sent to PostgreSQL
Insert	db.add(skill); db.commit()	INSERT INTO skills (...) VALUES (...) RETURNING id
Get by id	db.get(Skill, skill_id)	SELECT ... FROM skills WHERE skills.id = 7
Filter	select(User).where(User.email == data.email)	SELECT ... FROM users WHERE users.email = '...'
Search	Skill.title.ilike(f"%{q}%")	WHERE skills.title ILIKE '%html%'
Join	select(Skill).join(Skill.mentor)	FROM skills JOIN users ON users.id = skills.mentor_id
Update	setattr(skill, field, value); db.commit()	UPDATE skills SET level = ... WHERE id = 7
Delete	db.delete(skill); db.commit()	DELETE FROM skills WHERE skills.id = 9
Aggregate	func.avg(Review.rating) ... group_by(...)	SELECT avg(rating) ... GROUP BY mentor_id
Paging	.offset(skip).limit(limit)	LIMIT 20 OFFSET 0

Real SQL generated by SQLAlchemy from our skill search (`GET /skills?q=html&level=beginner`):

```
generated by SQLAlchemy

SELECT skills.id, skills.mentor_id, skills.title, skills.category, skills.level, skills.description, skills.
    created_at
FROM skills JOIN users ON users.id = skills.mentor_id
WHERE users.is_active IS true AND (skills.title ILIKE '%html%' OR skills.description ILIKE '%html%') AND skills.
    level = 'beginner' ORDER BY skills.id
 LIMIT 20 OFFSET 0
```

Real SQL for the mentor average rating (`rating_subquery` in users.py):

```
generated by SQLAlchemy

SELECT reviews.mentor_id, avg(reviews.rating) AS avg_rating, count(reviews.id) AS review_count
FROM reviews GROUP BY reviews.mentor_id
```

4.6 Input → output

One request, several queries: user + average rating + skills

REQUEST `GET /users/mentors/9` (no token)

RESPONSE 200 OK


```
{
  "id": 9,
  "name": "Ananya Rao",
  "bio": "MCA student, teaches HTML, CSS and JavaScript",
  "average_rating": 4.0,
  "review_count": 1,
  "skills": [
    {
      "id": 7,
      "mentor_id": 9,
      "title": "HTML & CSS Basics",
      "category": "Web Development",
      "level": "intermediate",
      "description": "HTML5 page structure, CSS flexbox and grid",
      "created_at": "2026-10-02T16:54:16.802468+05:30"
    },
    {
      "id": 8,
      "mentor_id": 9,
      "title": "JavaScript Fundamentals",
      "category": "Web Development",
      "level": "intermediate",
      "description": "Variables, functions, DOM and events",
      "created_at": "2026-10-02T16:54:16.808249+05:30"
    }
  ]
}
```


Say this when presenting

- “We use SQLAlchemy as our ORM: tables are Python classes and queries are Python code.”
- Show database.py: the engine and get_db, one session per request, always closed.
- Show the operation table and the real generated SQL. “The ORM also protects us from SQL injection.”

REQUIREMENT 5

Pydantic Validation

Group 2
Member 2

> **Expected implementation:** *Validate request data, required fields, data types, ranges, formats and business rules.*

5.1 What it means

Pydantic checks incoming data against rules we declare with Python type hints. FastAPI runs the check automatically **before** our endpoint code. Bad data never reaches the database.

5.2 How we used it in SkillSwap

- Every JSON body is a Pydantic class in `app/schemas.py` (UserRegister, SkillCreate, SessionCreate, ReviewCreate, …).
- Query and path parameters are checked with `Query(...)` and type hints (requirement 11).
- Our own rules are added with `@field_validator` (future date, strong password).
- Response classes (`UserOut`, `SkillOut` …) decide what is sent back — the password hash is never included.
- A failed check returns **422** with one entry per problem: where (`loc`), what (`msg`) and the error type.

5.3 All six kinds of check, with real results

Kind of check	Example field	Bad input	Real message
Required fields	UserRegister.name	(missing)	Field required
Data types	SessionCreate.skill_id: int	"abc"	Input should be a valid integer
Ranges	duration_minutes 30–180, rating 1–5	500 / 6	Input should be less than or equal to 180 / 5
Lengths	name 2–100, password 8–64	"X"	String should have at least 2 characters
Formats	email: EmailStr, scheduled_at: datetime	"bad", "tomorrow"	value is not a valid email address / valid datetime
Allowed values	role, level, status (enums)	"admin", "expert"	Input should be 'student' or 'mentor'
Business rules	future date, letter + digit in password	2024-01-01	scheduled_at must be in the future

5.4 The code

```
app/schemas.py — registration

class UserRegister(BaseModel):
    name: str = Field(min_length=2, max_length=100)
    email: EmailStr
    password: str = Field(min_length=8, max_length=64)
    role: Literal["student", "mentor"] = "student"  # admin cannot self-register
    bio: str | None = Field(default=None, max_length=500)

    @field_validator("password")
    @classmethod
    def strong_password(cls, v: str) -> str:
        if not any(c.isalpha() for c in v) or not any(c.isdigit() for c in v):
            raise ValueError("password must contain at least one letter and one digit")
        if len(v.encode()) > 72:
            raise ValueError("password is too long")
        return v
```

Code	What it means
`name: str = Field(min_length=2, max_length=100)`	Required (no default), text, 2–100 characters.
`email: EmailStr`	Must be a valid email address format.
`role: Literal["student", "mentor"] = "student"`	Only these two values; optional with default student.
`bio: str | None = Field(default=None, max_length=500)`	Optional; at most 500 characters.
`@field_validator("password")`	Our own rule: at least one letter and one digit, and not too long for bcrypt.
`raise ValueError(...)`	Pydantic turns this into a 422 entry with our message.

```
app/schemas.py — booking and reviews

class SessionCreate(BaseModel):
    skill_id: int = Field(gt=0)
    scheduled_at: datetime
    duration_minutes: int = Field(ge=30, le=180, description="30 to 180 minutes")
    notes: str | None = Field(default=None, max_length=500)

    @field_validator("scheduled_at")
    @classmethod
    def must_be_future(cls, v: datetime) -> datetime:
        v = as_utc(v)
        if v <= datetime.now(timezone.utc):
            raise ValueError("scheduled_at must be in the future")
        return v


class ReviewCreate(BaseModel):
    session_id: int = Field(gt=0)
    rating: int = Field(ge=1, le=5)
    comment: str | None = Field(default=None, max_length=500)
```

```
app/schemas.py — a response model (no password field)

class UserOut(ORMModel):
    id: int
    name: str
    email: EmailStr
    role: Role
    bio: str | None
    is_active: bool
    created_at: datetime
```

5.5 Every request body, field by field (from our OpenAPI schema)

UserRegister

Field	Type	Required	Rules
name	string	yes	min length 2, max length 100
email	string (email)	yes	—
password	string	yes	min length 8, max length 64
role	string	no	student | mentor, default "student"
bio	string	no	max length 500

UserLogin

Field	Type	Required	Rules
email	string (email)	yes	—
password	string	yes	min length 1

UserUpdate

Field	Type	Required	Rules
name	string	no	min length 2, max length 100
bio	string	no	max length 500

SkillCreate

Field	Type	Required	Rules
title	string	yes	min length 2, max length 100
category	string	yes	min length 2, max length 50
level	Level (enum)	yes	beginner | intermediate | advanced
description	string	no	max length 1000

SkillUpdate

Field	Type	Required	Rules
title	string	no	min length 2, max length 100
category	string	no	min length 2, max length 50
level	Level (enum)	no	beginner | intermediate | advanced
description	string	no	max length 1000

SessionCreate

Field	Type	Required	Rules
skill_id	integer	yes	> 0
scheduled_at	string (date-time)	yes	—
duration_minutes	integer	yes	≥ 30, ≤ 180
notes	string	no	max length 500

ReviewCreate

Field	Type	Required	Rules
session_id	integer	yes	> 0
rating	integer	yes	≥ 1, ≤ 5
comment	string	no	max length 500

ReviewUpdate

Field	Type	Required	Rules
rating	integer	no	≥ 1, ≤ 5
comment	string	no	max length 500

RoleUpdate

Field	Type	Required	Rules
role	Role (enum)	yes	student | mentor | admin

StatusUpdate

Field	Type	Required	Rules
is_active	boolean	yes	—

5.6 Input → output

Required field missing

REQUEST POST /auth/register (no token)

```
{
  "email": "x@skillswap.com",
  "password": "Abcd1234"
}
```

RESPONSE 422 Unprocessable Content

```
{
  "detail": [
    {
      "type": "missing",
      "loc": [
        "body",
        "name"
      ],
      "msg": "Field required"
    }
  ]
}
```

Several problems at once: length, format, allowed value

REQUEST POST /auth/register (no token)

```
{
  "name": "X",
  "email": "bad",
  "password": "short",
  "role": "admin"
}
```

RESPONSE 422 Unprocessable Content

```
{
  "detail": [
    {
      "type": "string_too_short",
      "loc": [
        "body",
        "name"
      ],
      "msg": "String should have at least 2 characters"
    },
    {
      "type": "value_error",
      "loc": [
        "body",
        "email"
      ],
      "msg": "value is not a valid email address: An email address must have an @-sign."
    },
    {
      "type": "string_too_short",
      "loc": [
        "body",
        "password"
      ],
      "msg": "String should have at least 8 characters"
    },
    {
      "type": "literal_error",
      "loc": [
        "body",
        "role"
      ],
      "msg": "Input should be 'student' or 'mentor'"
    }
  ]
}
```

Our own business rule: password must contain a digit

REQUEST POST /auth/register (no token)

```
{
  "name": "Weak Pass",
  "email": "weak@skillswap.com",
  "password": "onlyletters"
}
```

RESPONSE 422 Unprocessable Content

```
{
  "detail": [
    {
      "type": "value_error",
      "loc": [
        "body",
        "password"
      ],
      "msg": "Value error, password must contain at least one letter and one digit"
    }
  ]
}
```

Wrong data types and out-of-range value

REQUEST POST /sessions (Authorization: Bearer <token>)

```
{
  "skill_id": "abc",
  "scheduled_at": "tomorrow",
  "duration_minutes": 500
}
```

RESPONSE 422 Unprocessable Content

```
{
  "detail": [
    {
      "type": "int_parsing",
      "loc": [
        "body",
        "skill_id"
      ],
      "msg": "Input should be a valid integer, unable to parse string as an integer"
    },
    {
      "type": "datetime_from_date_parsing",
      "loc": [
        "body",
        "scheduled_at"
      ],
      "msg": "Input should be a valid datetime or date, input is too short"
    },
    {
      "type": "less_than_equal",
      "loc": [
        "body",
        "duration_minutes"
      ],
      "msg": "Input should be less than or equal to 180"
    }
  ]
}
```

Business rule in the schema: date must be in the future

REQUEST POST /sessions (Authorization: Bearer <token>)

```
{
  "skill_id": 7,
  "scheduled_at": "2024-01-01T10:00:00Z",
  "duration_minutes": 60
}
```

RESPONSE 422 Unprocessable Content

```
{
  "detail": [
    {
      "type": "value_error",
      "loc": [
        "body",
        "scheduled_at"
      ],
      "msg": "Value error, scheduled_at must be in the future"
    }
  ]
}
```

Range check: rating must be 1–5

REQUEST POST /reviews (Authorization: Bearer <token>)

```
{
  "session_id": 7,
  "rating": 6
}
```

RESPONSE 422 Unprocessable Content

```
{
  "detail": [
    {
      "type": "less_than_equal",
      "loc": [
        "body",
        "rating"
      ],
      "msg": "Input should be less than or equal to 5"
    }
  ]
}
```

Say this when presenting

- “Pydantic checks every request before our code runs; bad data gets 422 with a list of all problems.”
- Go through the six kinds of check with the real messages.
- Show UserRegister and SessionCreate, and point out our own validators.
- “Response models also protect us: UserOut has no password field.”

REQUIREMENT 6

JWT Authentication

Group 2
Member 2

> **Expected implementation:** *Implement registration/login and issue JWT access tokens.*

6.1 What it means

Authentication = proving who you are. After login the server gives a **JWT** (JSON Web Token): a signed text with your user id, role and expiry time. The client sends it with every request in the `Authorization: Bearer <token>` header. Because it is signed with our secret, it cannot be faked.

6.2 How we used it in SkillSwap

- `POST /auth/register`: create a student or mentor account (201). Email must be unique (409).
- `POST /auth/login`: email + password → `{"access_token", "token_type": "bearer", "role"}`.
- Tokens are made with **PyJWT**, algorithm **HS256**, secret `JWT_SECRET_KEY` from .env, valid for `ACCESS_TOKEN_EXPIRE_MINUTES` = 60 minutes.
- Wrong email or password → the same 401 message. Deactivated account → 403.
- Verifying the token on each request is done by `get_current_user` (requirement 10).

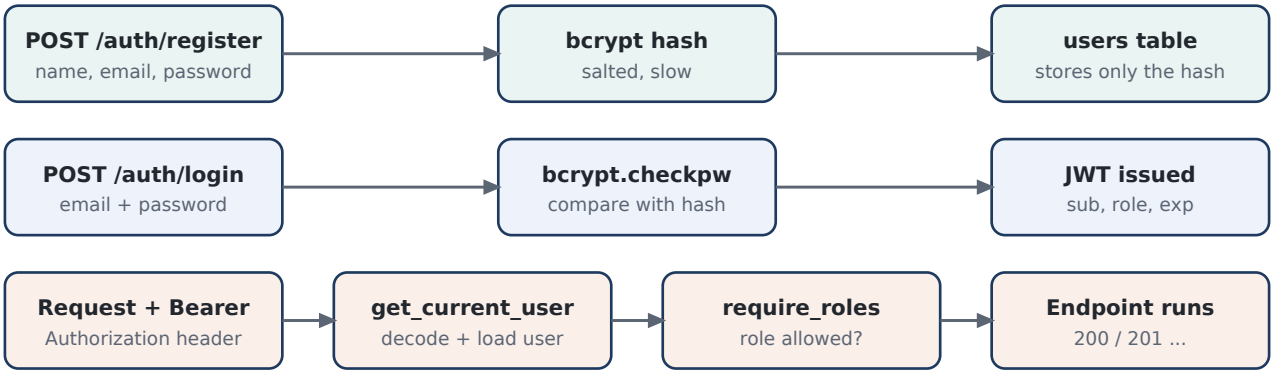


Figure. Register → login → using the token.

6.3 The code

app/routers/auth.py

```
@router.post("/register", response_model=UserOut, status_code=status.HTTP_201_CREATED)
def register(data: UserRegister, db: Session = Depends(get_db)):
    if db.scalar(select(User).where(User.email == data.email)):
        raise HTTPException(status.HTTP_409_CONFLICT, "Email is already registered")
    user = User(
        name=data.name,
        email=data.email,
        hashed_password=hash_password(data.password),
        role=data.role,
        bio=data.bio,
    )
    db.add(user)
    db.commit()
    db.refresh(user)
    return user


@router.post("/login", response_model=Token)
def login(data: UserLogin, db: Session = Depends(get_db)):
    user = db.scalar(select(User).where(User.email == data.email))
    if user is None or not verify_password(data.password, user.hashed_password):
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Invalid email or password")
    if not user.is_active:
        raise HTTPException(status.HTTP_403_FORBIDDEN, "Account is deactivated")
    return Token(access_token=create_access_token(user), role=user.role)
```

Code	What it means
`if db.scalar(select(User).where(User.email == data.email))`	Check the email is not taken; otherwise 409.
`hashed_password=hash_password(data.password)`	Store only the bcrypt hash (requirement 7).
`response_model=UserOut, status_code=201`	Return the new user without the password.
`verify_password(data.password, user.hashed_password)`	Compare the typed password with the stored hash.
`return Token(access_token=create_access_token(user), role=user.role)`	Send back the JWT and the role.

app/security.py

```
def create_access_token(user: User) -> str:
    expire = datetime.now(timezone.utc) + timedelta(minutes=settings.access_token_expire_minutes)
    payload = {"sub": str(user.id), "role": user.role, "exp": expire}
    return jwt.encode(payload, settings.jwt_secret_key, algorithm=settings.jwt_algorithm)
```

Code	What it means
`expire = now + timedelta(minutes=...)`	Expiry time: 60 minutes from now.
`payload = {"sub": str(user.id), "role": user.role, "exp": expire}`	What is inside the token: subject (user id), role, expiry.
`jwt.encode(payload, settings.jwt_secret_key, algorithm=...)`	Sign with the secret (HMAC-SHA256).

```
app/schemas.py — login input and output

class UserLogin(BaseModel):
    email: EmailStr
    password: str = Field(min_length=1)


class Token(BaseModel):
    access_token: str
    token_type: str = "bearer"
    role: Role
```

6.4 Inside a real token

```
Ananya's real access token, decoded

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
.eyJzdWIiOiI5Iiwicm9sZSI6Im1lbnRvciIsImV4cCI6MTc5MDk0Mzg1Nn0
.F-08xyKdrT2JuKFxYX3rhxbh_BZF4DRBaIJsOon2wMM

header    = {"alg": "HS256", "typ": "JWT"}
payload   = {"sub": "9", "role": "mentor", "exp": 1790943856}        # user 9 = Ananya, role mentor
exp       = 2026-10-02T12:24:16+00:00 (UTC)  → 60 minutes after login
signature = HMAC-SHA256(header + "." + payload, JWT_SECRET_KEY)
```

6.5 Input → output

Register a mentor

REQUEST POST /auth/register (no token)

```
{
  "name": "Ananya Rao",
  "email": "ananya.mentor@skillswap.com",
  "password": "Ananya123",
  "role": "mentor",
  "bio": "MCA student, 2 years of web development"
}
```

RESPONSE 201 Created

```
{
  "id": 9,
  "name": "Ananya Rao",
  "email": "ananya.mentor@skillswap.com",
  "role": "mentor",
  "bio": "MCA student, 2 years of web development",
  "is_active": true,
  "created_at": "2026-10-02T16:54:15.633358+05:30"
}
```

Register with an email that already exists

REQUEST POST /auth/register (no token)

```
{
  "name": "Kiran Again",
  "email": "kiran.student@skillswap.com",
  "password": "Kiran1234"
}
```

RESPONSE 409 Conflict

```
{
  "detail": "Email is already registered"
}
```

Login → JWT

REQUEST `POST /auth/login` (no token)

```
{
  "email": "ananya.mentor@skillswap.com",
  "password": "Ananya123"
}
```

RESPONSE 200 OK

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI...(shortened)",
  "token_type": "bearer",
  "role": "mentor"
}
```

Login with a wrong password

REQUEST `POST /auth/login` (no token)

```
{
  "email": "ananya.mentor@skillswap.com",
  "password": "Wrong1234"
}
```

RESPONSE 401 Unauthorized

```
{
  "detail": "Invalid email or password"
}
```

Say this when presenting

- “Register creates the account with a hashed password; login checks it and returns a signed JWT.”
- Show the decoded token: user id, role, expiry. “Anyone can read it, nobody can change it without our secret.”
- Show the 409 and 401 examples.

GROUP 3 · PRESENTED BY MEMBER 3 (name: _____)

Keeping accounts and data safe

Requirement 7 · Password Hashing
Requirement 8 · Authorization
Requirement 9 · RBAC (3 roles)

This group explains how passwords are stored safely, how we make sure users can only touch their own data, and how the three roles — student, mentor, admin — decide what each user may do.

REQUIREMENT 7

Password Hashing

Group 3
Member 3

> **Expected implementation:** *Never store plain-text passwords. Store securely hashed passwords.*

7.1 What it means

A **hash** is a one-way fingerprint of the password. You can check a password against it, but you cannot turn it back into the password. If the database is ever stolen, the real passwords stay secret.

7.2 How we used it in SkillSwap

- We use **bcrypt**, a password-hashing algorithm made to be slow, with cost factor **12** (2^12 rounds).
- bcrypt adds a random **salt**: the same password gives a different hash every time.
- Hashing happens in register and when the admin is created; checking happens in login.
- The hash is never returned by the API (`UserOut` has no password field).
- Passwords are limited to 72 bytes in our validator because bcrypt ignores anything longer.

7.3 The code

```
app/security.py

def hash_password(password: str) -> str:
    return bcrypt.hashpw(password.encode(), bcrypt.gensalt()).decode()


def verify_password(password: str, hashed: str) -> bool:
    return bcrypt.checkpw(password.encode(), hashed.encode())
```

Code	What it means
`bcrypt.gensalt()`	Make a new random salt (default cost 12).
`bcrypt.hashpw(password.encode(), salt)`	Hash the password bytes with the salt.
`bcrypt.checkpw(password.encode(), hashed.encode())`	Hash the typed password with the stored salt and compare. True or False.

7.4 Proof: what bcrypt really produces

```
run with our own security.py

hash_password("Kiran1234")  → $2b$12$W0JW/fFpVLyB3/UHkA51k.G12XbtPolmVqgXojp7sQAp/9Z23vexm
hash_password("Kiran1234")  → $2b$12$lzL7IDKYi.5xJiTevauil.TuQE4rrwQDmbXWWilMJXiDiTLperJO.     # same password,
    different hash (random salt)

verify_password("Kiran1234", first_hash)  → True
verify_password("kiran1234", first_hash)  → False       # one capital letter different

$2b$ 12 $ W0JW/fFpVLyB3/UHkA51k. G12XbtPolmVqgXojp7sQAp/9Z23vexm
 │    │   └ salt (22 chars) ────┘ └ hash (31 chars) ──────────────┘
 │    └ cost factor: 2^12 rounds
 └ bcrypt version
```

```
what PostgreSQL really stores

skillswap=# SELECT id, email, left(hashed_password, 30) || '...' AS hashed_password FROM users WHERE id IN (1,
    9, 10);
 id |            email            |          hashed_password
----+-----------------------------+----------------------------------
  1 | admin@skillswap.com         | $2b$12$3/FRYcQ0U8evrZkwhKGOpu3...
  9 | ananya.mentor@skillswap.com | $2b$12$C9ymKJPcja1Wd.x3dAjjb.V...
 10 | kiran.student@skillswap.com | $2b$12$FF7MhpyD2n4oSmT.LJfSK.K...
(3 rows)
```

7.5 Input → output

The password goes in …

REQUEST `POST /auth/register` (no token)

```
{
  "name": "Kiran Kumar",
  "email": "kiran.student@skillswap.com",
  "password": "Kiran1234",
  "role": "student"
}
```

RESPONSE 201 Created

```
{
  "id": 10,
  "name": "Kiran Kumar",
  "email": "kiran.student@skillswap.com",
  "role": "student",
  "bio": null,
  "is_active": true,
  "created_at": "2026-10-02T16:54:15.834732+05:30"
}
```

… and nothing about the password comes out. Login then works by comparing hashes:

A wrong password does not match the hash

REQUEST `POST /auth/login` (no token)

```
{
  "email": "ananya.mentor@skillswap.com",
  "password": "Wrong1234"
}
```

RESPONSE 401 Unauthorized

```
{
  "detail": "Invalid email or password"
}
```

Say this when presenting

- “We never store passwords. bcrypt turns them into a salted, slow, one-way hash.”
- Show the two different hashes of the same password, and checkpw returning True/False.
- Show the users table in pgAdmin: only $2b$12$… values.

REQUIREMENT 8

Authorization

Group 3
Member 3

> **Expected implementation:** *Protect APIs based on the authenticated user's permissions.*

8.1 What it means

Authentication asks "**who are you?**". Authorization asks "**are you allowed to do this?**". Even a logged-in user may only touch the data they are allowed to.

8.2 How we used it in SkillSwap

We check permissions at three levels:

- **Role level** — which kind of user may call an endpoint (requirement 9).
- **Ownership level** — is this **your** skill / session / review?
- **Data level** — lists only contain what you may see, and owner ids come from the token, never from the request body.

Permission rule	Where in the code	Result if broken
Update / delete a skill: its mentor or an admin	skills.py · check_owner()	403 You can only modify your own skills
See / cancel / complete a session: its student, its mentor or an admin	sessions.py · get_session_for_user()	403 This is not your session
Review a session: its own student, and only after it is completed	reviews.py · create_review()	403 / 400
Edit a review: its author	reviews.py · update_review()	403 You can only edit your own reviews
Delete a review: its author or an admin	reviews.py · delete_review()	403 You can only delete your own reviews
Session history: only your own (admin: all)	sessions.py · session_history()	others are filtered out
New skill belongs to the caller	skills.py · mentor_id=user.id	cannot create for others
New session belongs to the caller	sessions.py · student_id=user.id	cannot book for others
Deactivated users are blocked	auth.py · login, security.py	403 at login, 401 with old token

8.3 The code

app/routers/skills.py

```
def check_owner(skill: Skill, user: User):
    if user.role != Role.ADMIN and skill.mentor_id != user.id:
        raise HTTPException(status.HTTP_403_FORBIDDEN, "You can only modify your own skills")
```

app/routers/sessions.py

```
def get_session_for_user(session_id: int, user: User, db: Session) -> MentoringSession:
    """Return the session if it exists (404) and the user takes part in it or is admin (403)."""
    session = db.get(MentoringSession, session_id)
    if session is None:
        raise HTTPException(status.HTTP_404_NOT_FOUND, "Session not found")
    if user.role != Role.ADMIN and user.id not in (session.student_id, session.mentor_id):
        raise HTTPException(status.HTTP_403_FORBIDDEN, "This is not your session")
    return session
```

Code	What it means
`if user.role != Role.ADMIN and skill.mentor_id != user.id`	Not an admin and not the owner → stop.
`raise HTTPException(status.HTTP_403_FORBIDDEN, ...)`	Answer 403 Forbidden with a clear message.
`user.id not in (session.student_id, session.mentor_id)`	Only the two people in the session (or an admin) may see or change it.

`app/routers/sessions.py · session_history — data-level filtering`

```
stmt = select(MentoringSession)
if user.role != Role.ADMIN:
    stmt = stmt.where(or_(MentoringSession.student_id == user.id, MentoringSession.mentor_id == user.id))
```

8.4 Input → output

Kiran (student) tries to change Ananya's skill

REQUEST `PUT /skills/7` (Authorization: Bearer <token>)

```
{
  "title": "Hacked"
}
```

RESPONSE 403 Forbidden

```
{
  "detail": "You do not have permission to perform this action"
}
```

Ananya tries to edit Kiran's review of her

REQUEST `PUT /reviews/4` (Authorization: Bearer <token>)

```
{
  "rating": 1
}
```

RESPONSE 403 Forbidden

```
{
  "detail": "You can only edit your own reviews"
}
```

Ananya only sees her own sessions

REQUEST `GET /sessions?status=booked` (Authorization: Bearer <token>)

RESPONSE 200 OK

```
[
  {
    "id": 7,
    "student_id": 10,
    "mentor_id": 9,
    "skill_id": 7,
    "scheduled_at": "2026-10-06T14:30:00+05:30",
    "duration_minutes": 60,
    "status": "booked",
    "notes": "Please help me build a portfolio page",
    "created_at": "2026-10-02T16:54:16.859910+05:30"
  }
]
```

Say this when presenting

- “Login tells us who you are; authorization decides what you may touch.”
- Go through the permission table, then show check_owner and get_session_for_user.
- Show the two 403 examples: a student editing a mentor’s skill, a mentor editing a student’s review.

REQUIREMENT 9

RBAC — Role-Based Access Control

Group 3
Member 3

> **Expected implementation:** *Implement at least 3 roles where applicable, such as User/Student, Staff/Organizer and Admin.*

9.1 What it means

With RBAC every user has a **role**, and each endpoint says which roles may use it. Permissions are given to roles, not to individual people.

9.2 How we used it in SkillSwap

Required kind of role	Our role	How the account gets it
User / Student	student	register with role “student” (default)
Staff / Organizer	mentor — organises and teaches sessions	register with role “mentor”
Admin	admin	created on startup from ADMIN_EMAIL / ADMIN_PASSWORD; nobody can register as admin

- The role is a column in `users` and is put into the JWT.
- One reusable dependency `require_roles(...)` enforces it. Wrong role → **403**.
- The admin can change anyone’s role (`PATCH /admin/users/{id}/role`), but not their own.

9.3 The code

app/models.py

```
class Role(StrEnum):
    STUDENT = "student"
    MENTOR = "mentor"
    ADMIN = "admin"
```

app/security.py

```
def require_roles(*roles: Role):
    """Dependency factory for role-based access control (403 if role not allowed)."""

    def checker(user: User = Depends(get_current_user)) -> User:
        if user.role not in roles:
            raise HTTPException(status.HTTP_403_FORBIDDEN, "You do not have permission to perform this action")
        return user

    return checker
```

Code	What it means
`def require_roles(*roles: Role)`	A dependency factory: we pass the roles that are allowed.
`def checker(user = Depends(get_current_user))`	First make sure the user is logged in (401 if not).
`if user.role not in roles: raise ... 403`	Logged in but the wrong role → 403.
`return checker`	FastAPI runs this checker before the endpoint.

```
where roles are enforced

# skills.py
def create_skill(data: SkillCreate, user: User = Depends(require_roles(Role.MENTOR)), ...)
def update_skill(skill_id: int, data: SkillUpdate, user: User = Depends(require_roles(Role.MENTOR, Role.ADMIN)),
     ...)
# sessions.py
def book_session(data: SessionCreate, user: User = Depends(require_roles(Role.STUDENT)), ...)
def complete_session(session_id: int, user: User = Depends(require_roles(Role.MENTOR, Role.ADMIN)), ...)
# reviews.py
def create_review(data: ReviewCreate, user: User = Depends(require_roles(Role.STUDENT)), ...)
# admin.py — the whole router at once
router = APIRouter(prefix="/admin", tags=["Admin"], dependencies=[Depends(require_roles(Role.ADMIN))])
```

9.4 Permission matrix

Action	Public	Student	Mentor	Admin
Register, login, search skills/mentors, read reviews	✔	✔	✔	✔
View / update own profile	—	✔	✔	✔
Create a skill	—	—	✔	—
Update / delete a skill	—	—	own	any
Book a session	—	✔	—	—
See sessions	—	own	own	all
Cancel a session	—	own	own	any
Complete a session	—	—	own	any
Write a review	—	own completed	—	—
Users, roles, status, delete, stats (/admin)	—	—	—	✔

9.5 Input → output

A student calls a mentor-only endpoint

REQUEST `POST /skills` (Authorization: Bearer <token>)

```
{
  "title": "Chess",
  "category": "Games",
  "level": "beginner"
}
```

RESPONSE 403 Forbidden

```
{
  "detail": "You do not have permission to perform this action"
}
```

A mentor calls a student-only endpoint

REQUEST `POST /sessions` (Authorization: Bearer <token>)

```
{
  "skill_id": 7,
  "scheduled_at": "2026-10-07T09:00:00Z",
  "duration_minutes": 60
}
```

RESPONSE 403 Forbidden

```
{
  "detail": "You do not have permission to perform this action"
}
```

A student calls an admin endpoint

REQUEST `GET /admin/users` (Authorization: Bearer <token>)

RESPONSE 403 Forbidden


```
{
  "detail": "You do not have permission to perform this action"
}
```


The admin calls it: active mentors

REQUEST `GET /admin/users?role=mentor&is_active=true` (Authorization: Bearer <token>)

RESPONSE 200 OK


```
[
  {
    "id": 7,
    "name": "Priya Sharma",
    "email": "priya.mentor@skillswap.com",
    "role": "mentor",
    "bio": "Final-year CS student, 3 years of Python experience",
    "is_active": true,
    "created_at": "2026-10-02T13:07:01.278328+05:30"
  },
  {
    "id": 9,
    "name": "Ananya Rao",
    "email": "ananya.mentor@skillswap.com",
    "role": "mentor",
    "bio": "MCA student, teaches HTML, CSS and JavaScript",
    "is_active": true,
    "created_at": "2026-10-02T16:54:15.633358+05:30"
  }
]
```


The admin changes a user's role

REQUEST `PATCH /admin/users/11/role` (Authorization: Bearer <token>)


```
{
  "role": "mentor"
}
```


RESPONSE 200 OK


```
{
  "id": 11,
  "name": "Temp User",
  "email": "temp.user@skillswap.com",
  "role": "mentor",
  "bio": null,
  "is_active": true,
  "created_at": "2026-10-02T16:54:16.960433+05:30"
}
```


Say this when presenting

- "We have the three roles the requirement asks for: student, mentor (our staff/organizer) and admin."
- Show require_roles and how one line protects the whole admin router.
- Show the permission matrix and the three 403 examples, one per role.

GROUP 4 · PRESENTED BY MEMBER 4 (name: _____)

Guarding routes and designing URLs

Requirement 10 · Protected Routes
Requirement 11 · Path & Query Parameters
Requirement 12 · HTTP Status Codes

This group explains how FastAPI dependencies find the current user and protect private endpoints, how URLs carry ids and search filters, and which status code our API answers in every situation.

REQUIREMENT 10

Protected Routes

Group 4 **Member 4**

Expected implementation: *Use FastAPI dependencies to identify the current user and protect private endpoints.*

10.1 What it means

A **dependency** is a function FastAPI runs **before** an endpoint. If it fails, the endpoint never runs. We use one dependency to read the token and find the current user.

10.2 How we used it in SkillSwap

- `get_current_user` reads the `Authorization: Bearer` header with FastAPI's `HTTPBearer`, decodes the JWT, loads the user from PostgreSQL and returns it.
- Any endpoint becomes private by adding `Depends(get_current_user)` or `Depends(require_roles(...))`.
- **20 of 29 endpoints are protected**. Public: register, login, search skills, get skill, search mentors, mentor profile, list reviews, get review, health.
- No token, a fake token, an expired token, or a deactivated / deleted user → **401**.
- Because the user is loaded from the database on every request, deactivating a user blocks them **immediately**.

10.3 The code

app/security.py

```
bearer_scheme = HTTPBearer(auto_error=False)


def get_current_user(
    credentials: HTTPAuthorizationCredentials | None = Depends(bearer_scheme),
    db: Session = Depends(get_db),
) -> User:
    """Dependency that reads the Bearer token and returns the logged-in user (401 otherwise)."""
    unauthorized = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Invalid or missing token",
        headers={"WWW-Authenticate": "Bearer"},
    )
    if credentials is None:
        raise unauthorized
    try:
        payload = jwt.decode(credentials.credentials, settings.jwt_secret_key, algorithms=[settings.
    jwt_algorithm])
        user_id = int(payload["sub"])
    except (jwt.PyJWTError, KeyError, ValueError):
        raise unauthorized
    user = db.get(User, user_id)
    if user is None or not user.is_active:
        raise unauthorized
    return user
```

Code	What it means
`HTTPBearer(auto_error=False)`	Reads "Authorization: Bearer ..."; also adds the Authorize button in Swagger. We return our own 401 if missing.
`credentials = Depends(bearer_scheme)`	The token text, or None.
`jwt.decode(..., algorithms=[...])`	Checks the signature and the expiry. Any problem raises an error.

Code	What it means
`except (jwt.PyJWTError, KeyError, ValueError)`	Fake, expired or malformed token → 401.
`db.get(User, user_id)`	Load the user from PostgreSQL.
`if user is None or not user.is_active`	Deleted or deactivated user → 401.
`return user`	The endpoint receives the current user as a Python object.

app/routers/users.py — protected endpoints

```
@router.get("/me", response_model=UserOut)
def get_me(user: User = Depends(get_current_user)):
    return user


@router.put("/me", response_model=UserOut)
def update_me(data: UserUpdate, user: User = Depends(get_current_user), db: Session = Depends(get_db)):
    for field, value in data.model_dump(exclude_unset=True).items():
        setattr(user, field, value)
    db.commit()
    db.refresh(user)
    return user
```

10.4 Protected vs public endpoints

Protected (lock in Swagger)	Public
GET/PUT /users/me · POST /skills · PUT/DELETE /skills/{id} · all /sessions · POST /reviews · PUT/DELETE /reviews/{id} · all 7 /admin endpoints	POST /auth/register · POST /auth/login · GET /skills · GET /skills/{id} · GET /users/mentors · GET /users/mentors/{id} · GET /reviews · GET /reviews/{id} · GET /

10.5 Input → output

No token

REQUEST GET /users/me (no token)

RESPONSE 401 Unauthorized

```
{
  "detail": "Invalid or missing token"
}
```

A fake token

REQUEST GET /users/me (Authorization: Bearer <token>)

RESPONSE 401 Unauthorized

```
{
  "detail": "Invalid or missing token"
}
```

A valid token: the dependency found Ananya

REQUEST GET /users/me (Authorization: Bearer <token>)

RESPONSE 200 OK

```
{
  "id": 9,
  "name": "Ananya Rao",
  "email": "ananya.mentor@skillswap.com",
  "role": "mentor",
  "bio": "MCA student, 2 years of web development",
  "is_active": true,
  "created_at": "2026-10-02T16:54:15.633358+05:30"
}
```

A deactivated user cannot log in

REQUEST `POST /auth/login` (no token)

```
{
  "email": "temp.user@skillswap.com",
  "password": "Temp12345"
}
```

RESPONSE 403 Forbidden

```
{
  "detail": "Account is deactivated"
}
```

Say this when presenting

- “get_current_user is a FastAPI dependency: it runs first, checks the token, and gives the endpoint the current user.”
- Show the three cases: no token 401, fake token 401, valid token 200.
- “20 of our 29 endpoints are protected; in Swagger they show a lock.”

REQUIREMENT 11

Path & Query Parameters

Group 4
Member 4

> **Expected implementation:** *Use path parameters and query parameters for resource lookup, searching and filtering.*

11.1 What it means

A **path parameter** is part of the URL path and picks **one** resource: `/skills/7`. A **query parameter** comes after `?` and **filters or searches** a list: `/skills?q=html&level=beginner`. Several query parameters are joined with `&`.

11.2 How we used it in SkillSwap

- **Path parameters for lookup**: skill_id, session_id, review_id, mentor_id, user_id — always integers; a wrong type gives 422, an unknown id 404.
- **Query parameters for searching and filtering** on 5 list endpoints: text search (`q`, `skill`), filters (`category`, `level`, `status`, `role`, `is_active`, `mentor_id`), ranges (`min_rating`, `from_date`, `to_date`).
- **Paging** on every list: `skip` (how many to skip) and `limit` (how many to return, 1–100, default 20).
- All parameters are optional and validated; text search is case-insensitive (`ILIKE`).

11.3 Every parameter in our API (from the OpenAPI schema)

Endpoint	Parameter	In	Type	Rules / default
GET /users/mentors	skill	query	string	min len 1
GET /users/mentors	category	query	string	min len 1
GET /users/mentors	min_rating	query	number	≥ 1, ≤ 5
GET /users/mentors	skip	query	integer	≥ 0, default 0
GET /users/mentors	limit	query	integer	≥ 1, ≤ 100, default 20
GET /users/mentors/{mentor_id}	mentor_id	path	integer	required
GET /skills	q	query	string	min len 1
GET /skills	category	query	string	min len 1
GET /skills	level	query	Level	beginner | intermediate | advanced
GET /skills	mentor_id	query	integer	> 0
GET /skills	skip	query	integer	≥ 0, default 0
GET /skills	limit	query	integer	≥ 1, ≤ 100, default 20
GET /skills/{skill_id}	skill_id	path	integer	required
PUT /skills/{skill_id}	skill_id	path	integer	required
DELETE /skills/{skill_id}	skill_id	path	integer	required
GET /sessions	status	query	SessionStatus	booked | completed | cancelled
GET /sessions	from_date	query	string (date-time)	optional
GET /sessions	to_date	query	string (date-time)	optional
GET /sessions	skip	query	integer	≥ 0, default 0
GET /sessions	limit	query	integer	≥ 1, ≤ 100, default 20
GET /sessions/{session_id}	session_id	path	integer	required

Endpoint	Parameter	In	Type	Rules / default
PATCH /sessions/{session_id}/cancel	session_id	path	integer	required
PATCH /sessions/{session_id}/complete	session_id	path	integer	required
GET /reviews	mentor_id	query	integer	> 0
GET /reviews	min_rating	query	integer	≥ 1, ≤ 5
GET /reviews	skip	query	integer	≥ 0, default 0
GET /reviews	limit	query	integer	≥ 1, ≤ 100, default 20
GET /reviews/{review_id}	review_id	path	integer	required
PUT /reviews/{review_id}	review_id	path	integer	required
DELETE /reviews/{review_id}	review_id	path	integer	required
GET /admin/users	role	query	Role	student | mentor | admin
GET /admin/users	is_active	query	boolean	optional
GET /admin/users	q	query	string	min len 1
GET /admin/users	skip	query	integer	≥ 0, default 0
GET /admin/users	limit	query	integer	≥ 1, ≤ 100, default 20
GET /admin/users/{user_id}	user_id	path	integer	required
DELETE /admin/users/{user_id}	user_id	path	integer	required
PATCH /admin/users/{user_id}/role	user_id	path	integer	required
PATCH /admin/users/{user_id}/status	user_id	path	integer	required
DELETE /admin/sessions/{session_id}	session_id	path	integer	required

11.4 The code

```
app/routers/skills.py — search and filter

@router.get("", response_model=list[SkillOut])
def search_skills(
    q: str | None = Query(None, min_length=1, description="Search in title and description"),
    category: str | None = Query(None, min_length=1),
    level: Level | None = None,
    mentor_id: int | None = Query(None, gt=0),
    skip: int = Query(0, ge=0),
    limit: int = Query(20, ge=1, le=100),
    db: Session = Depends(get_db),
):
    stmt = select(Skill).join(Skill.mentor).where(User.is_active.is_(True))
    if q:
        stmt = stmt.where(or_(Skill.title.ilike(f"%{q}%"), Skill.description.ilike(f"%{q}%")))
    if category:
        stmt = stmt.where(Skill.category.ilike(category))
    if level:
        stmt = stmt.where(Skill.level == level)
    if mentor_id:
        stmt = stmt.where(Skill.mentor_id == mentor_id)
    return db.scalars(stmt.order_by(Skill.id).offset(skip).limit(limit)).all()
```

Code	What it means
`q: str | None = Query(None, min_length=1, description=...)`	Optional ?q=...; if sent it cannot be empty; the description appears in Swagger.
`level: Level | None = None`	Optional ?level=..., must be one of the Level enum values.
`mentor_id: int | None = Query(None, gt=0)`	Optional filter by mentor; must be positive.

Code	What it means
`skip: int = Query(0, ge=0), limit: int = Query(20, ge=1, le=100)`	Paging with safe limits.
`if q: stmt = stmt.where(or_(... ilike ...))`	Each filter is added only if it was sent.

```
app/routers/sessions.py — filtering by status and dates

@router.get("", response_model=list[SessionOut])
def session_history(
    status_filter: SessionStatus | None = Query(None, alias="status"),
    from_date: datetime | None = Query(None, description="Only sessions on/after this time"),
    to_date: datetime | None = Query(None, description="Only sessions on/before this time"),
    skip: int = Query(0, ge=0),
    limit: int = Query(20, ge=1, le=100),
    user: User = Depends(get_current_user),
    db: Session = Depends(get_db),
):
    """Students/mentors see their own sessions; admins see all sessions."""
    stmt = select(MentoringSession)
    if user.role != Role.ADMIN:
        stmt = stmt.where(or_(MentoringSession.student_id == user.id, MentoringSession.mentor_id == user.id))
    if status_filter:
        stmt = stmt.where(MentoringSession.status == status_filter)
    if from_date:
        stmt = stmt.where(MentoringSession.scheduled_at >= as_utc(from_date))
    if to_date:
        stmt = stmt.where(MentoringSession.scheduled_at <= as_utc(to_date))
    stmt = stmt.order_by(MentoringSession.scheduled_at.desc()).offset(skip).limit(limit)
    return db.scalars(stmt).all()
```

Code	What it means
`Query(None, alias="status")`	The URL name is ?status=..., the Python name is status_filter.
`from_date / to_date: datetime | None`	Optional date range; must be a valid date-time.

11.5 Input → output

Search + two filters

REQUEST `GET /skills?q=html&category=Web%20Development&level=beginner` (no token)

RESPONSE 200 OK

```
[
  {
    "id": 7,
    "mentor_id": 9,
    "title": "HTML & CSS Basics",
    "category": "Web Development",
    "level": "beginner",
    "description": "Build your first web page with HTML and style it with CSS",
    "created_at": "2026-10-02T16:54:16.802468+05:30"
  }
]
```

Mentors teaching HTML with an average rating of at least 4

REQUEST `GET /users/mentors?skill=html&min_rating=4` (no token)

RESPONSE 200 OK

```
[
  {
    "id": 9,
    "name": "Ananya Rao",
    "bio": "MCA student, teaches HTML, CSS and JavaScript",
    "average_rating": 4.0,
    "review_count": 1
  }
]
```

Kiran’s history since 1 Jan 2026 (date filter)

REQUEST `GET /sessions?from_date=2026-01-01T00:00:00Z` (Authorization: Bearer <token>)

RESPONSE 200 OK


```
[
  {
    "id": 8,
    "student_id": 10,
    "mentor_id": 9,
    "skill_id": 7,
    "scheduled_at": "2026-10-07T14:30:00+05:30",
    "duration_minutes": 90,
    "status": "cancelled",
    "notes": null,
    "created_at": "2026-10-02T16:54:16.893792+05:30"
  },
  {
    "id": 7,
    "student_id": 10,
    "mentor_id": 9,
    "skill_id": 7,
    "scheduled_at": "2026-10-06T14:30:00+05:30",
    "duration_minutes": 60,
    "status": "completed",
    "notes": "Please help me build a portfolio page",
    "created_at": "2026-10-02T16:54:16.859910+05:30"
  }
]
```


Reviews for one mentor, rating ≥ 4

REQUEST `GET /reviews?mentor_id=9&min_rating=4` (no token)

RESPONSE 200 OK


```
[
  {
    "id": 4,
    "session_id": 7,
    "student_id": 10,
    "mentor_id": 9,
    "rating": 4,
    "comment": "Clear explanations, a bit fast",
    "created_at": "2026-10-02T16:54:16.923724+05:30"
  }
]
```


Query parameters are validated too

REQUEST `GET /skills?limit=500` (no token)

RESPONSE 422 Unprocessable Content


```
{
  "detail": [
    {
      "type": "less_than_equal",
      "loc": [
        "query",
        "limit"
      ],
      "msg": "Input should be less than or equal to 100"
    }
  ]
}
```

A path parameter: one session by id

REQUEST `GET /sessions/7` (Authorization: Bearer <token>)

RESPONSE 200 OK


```
{
  "id": 7,
  "student_id": 10,
  "mentor_id": 9,
  "skill_id": 7,
  "scheduled_at": "2026-10-06T14:30:00+05:30",
  "duration_minutes": 60,
  "status": "booked",
  "notes": "Please help me build a portfolio page",
  "created_at": "2026-10-02T16:54:16.859910+05:30"
}
```


Say this when presenting

- “Path parameters pick one thing: /skills/7. Query parameters search and filter lists: /skills?q=html&level;=beginner.”
- Show the full parameter table and search_skills: every filter is optional and added only if sent.
- Show the search, rating filter, date filter and the 422 for limit=500.

REQUIREMENT 12

HTTP Status Codes

Group 4
Member 4

> **Expected implementation:** *Use appropriate status codes such as 200, 201, 400, 401, 403, 404 and 409.*

12.1 What it means

Every HTTP response has a 3-digit **status code** that tells the client what happened. **2xx** = success, **4xx** = the client did something wrong, **5xx** = the server failed.

12.2 How we used it in SkillSwap

Code	Meaning	When SkillSwap returns it	Real example in this document
200 OK	success	read, update, login, cancel, complete	login_mentor, skill_update
201 Created	new record created	register, create skill, book session, create review	register_mentor, skill_create
204 No Content	deleted, empty body	delete skill, review, user, session	skill_delete
400 Bad Request	valid data, rule says no	cancel a non-booked session, review a cancelled one, delete a skill with upcoming sessions, admin changing themselves	session_cancel_again
401 Unauthorized	not authenticated	no / fake / expired token, wrong password	me_no_token, login_wrong
403 Forbidden	not allowed	wrong role, not the owner, deactivated at login	skill_student_403
404 Not Found	does not exist	unknown skill, session, review, user or mentor id	skill_404
409 Conflict	duplicate / clash	email taken, duplicate skill title, double booking, second review	session_overlap
422 Unprocessable	invalid data	Pydantic validation of body, path and query	register_invalid

12.3 The code

Success codes are set on the route decorator; error codes are raised. We use the named constants from `fastapi.status` so the meaning is clear in the code:

```
pattern used in every router

from fastapi import HTTPException, status

@router.post("", response_model=SessionOut, status_code=status.HTTP_201_CREATED)      # success code
...
    raise HTTPException(status.HTTP_404_NOT_FOUND, "Skill not found")               # error codes
    raise HTTPException(status.HTTP_400_BAD_REQUEST, "This mentor is not available")
    raise HTTPException(status.HTTP_409_CONFLICT, "Mentor already has a session booked at this time")

@router.delete("/{skill_id}", status_code=status.HTTP_204_NO_CONTENT)
...
    return Response(status_code=status.HTTP_204_NO_CONTENT)
```

12.4 The status codes of every endpoint

Produced automatically from our source code (the routes, their dependencies and every `raise HTTPException`):

Method	Path	Access	Success	Possible error codes
POST	/auth/register	public	201	409, 422
POST	/auth/login	public	200	401, 403, 422
GET	/users/me	logged in	200	401
PUT	/users/me	logged in	200	401, 422
GET	/users/mentors	public	200	422
GET	/users/mentors/{mentor_id}	public	200	404, 422
POST	/skills	mentor	201	401, 403, 409, 422
GET	/skills	public	200	422
GET	/skills/{skill_id}	public	200	404, 422
PUT	/skills/{skill_id}	mentor, admin	200	401, 403, 404, 409, 422
DELETE	/skills/{skill_id}	mentor, admin	204	400, 401, 403, 404, 422
POST	/sessions	student	201	400, 401, 403, 404, 409, 422
GET	/sessions	logged in	200	401, 422
GET	/sessions/{session_id}	logged in	200	401, 403, 404, 422
PATCH	/sessions/{session_id}/cancel	logged in	200	400, 401, 403, 404, 422
PATCH	/sessions/{session_id}/complete	mentor, admin	200	400, 401, 403, 404, 422
POST	/reviews	student	201	400, 401, 403, 404, 409, 422
GET	/reviews	public	200	422
GET	/reviews/{review_id}	public	200	404, 422
PUT	/reviews/{review_id}	logged in	200	401, 403, 404, 422
DELETE	/reviews/{review_id}	logged in	204	401, 403, 404, 422
GET	/admin/users	admin	200	401, 403, 422
GET	/admin/users/{user_id}	admin	200	401, 403, 404, 422
PATCH	/admin/users/{user_id}/role	admin	200	400, 401, 403, 404, 422
PATCH	/admin/users/{user_id}/status	admin	200	400, 401, 403, 404, 422
DELETE	/admin/users/{user_id}	admin	204	400, 401, 403, 404, 422
DELETE	/admin/sessions/{session_id}	admin	204	401, 403, 404, 422
GET	/admin/stats	admin	200	401, 403
GET	/	public	200	—

12.5 Input → output: one real example of every code

200 OK

REQUEST `POST /auth/login` (no token)

```
{
  "email": "ananya.mentor@skillswap.com",
  "password": "Ananya123"
}
```

RESPONSE 200 OK

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI...(shortened)",
  "token_type": "bearer",
  "role": "mentor"
}
```

201 Created

REQUEST `POST /skills` (Authorization: Bearer <token>)

```
{
  "title": "JavaScript Fundamentals",
  "category": "Web Development",
  "level": "intermediate",
  "description": "Variables, functions, DOM and events"
}
```

RESPONSE 201 Created

```
{
  "id": 8,
  "mentor_id": 9,
  "title": "JavaScript Fundamentals",
  "category": "Web Development",
  "level": "intermediate",
  "description": "Variables, functions, DOM and events",
  "created_at": "2026-10-02T16:54:16.808249+05:30"
}
```

204 No Content

REQUEST `DELETE /skills/9` (Authorization: Bearer <token>)

RESPONSE 204 No Content

```
(empty body)
```

400 Bad Request

REQUEST `PATCH /sessions/8/cancel` (Authorization: Bearer <token>)

RESPONSE 400 Bad Request

```
{
  "detail": "Cannot cancel a cancelled session"
}
```

401 Unauthorized

REQUEST `GET /users/me` (no token)

RESPONSE 401 Unauthorized

```
{
  "detail": "Invalid or missing token"
}
```

403 Forbidden

REQUEST `POST /skills` (Authorization: Bearer <token>)

```
{
  "title": "Chess",
  "category": "Games",
  "level": "beginner"
}
```

RESPONSE 403 Forbidden

```
{
  "detail": "You do not have permission to perform this action"
}
```

404 Not Found

REQUEST `GET /skills/999999` (no token)

RESPONSE 404 Not Found

```
{
  "detail": "Skill not found"
}
```

409 Conflict

REQUEST `POST /sessions` (Authorization: Bearer <token>)

```
{
  "skill_id": 7,
  "scheduled_at": "2026-10-06T09:30:00Z",
  "duration_minutes": 30
}
```

RESPONSE 409 Conflict

```
{
  "detail": "Mentor already has a session booked at this time"
}
```

422 Unprocessable Content

REQUEST `POST /reviews` (Authorization: Bearer <token>)

```
{
  "session_id": 7,
  "rating": 6
}
```

RESPONSE 422 Unprocessable Content

```
{
  "detail": [
    {
      "type": "less_than_equal",
      "loc": [
        "body",
        "rating"
      ],
      "msg": "Input should be less than or equal to 5"
    }
  ]
}
```

Say this when presenting

- “We use every required code — 200, 201, 400, 401, 403, 404, 409 — plus 204 for deletes and 422 for invalid data.”
- Show the meaning table, then the per-endpoint table generated from our code.
- Pick 3–4 of the real examples: 201, 401 vs 403, 409.

GROUP 5 · PRESENTED BY MEMBER 5 (name: _____)

Handling errors, configuration, docs and testing

Requirement 13 · Exception Handling
Requirement 14 · Environment Variables
Requirement 15 · Swagger / OpenAPI
Requirement 16 · Postman Testing

This group explains how every kind of error is turned into a clear answer, how secrets are kept out of the code, how the API documents itself in Swagger, and how we tested all of it with a Postman collection.

REQUIREMENT 13

Exception Handling

Group 5
Member 5

> **Expected implementation:** *Handle invalid input, missing resources, duplicate records and business-rule failures.*

13.1 What it means

When something goes wrong, the API must not crash or return a confusing 500 error. It should stop, and answer with the right status code and a clear message.

13.2 How we used it in SkillSwap

Required case	Our technique	Code	Example message
Invalid input	Pydantic validation, automatic	schemas.py + type hints	Field required
Missing resources	helper functions that raise 404	get_skill_or_404, get_review_or_404, get_user_or_404, get_session_for_user	Skill not found
Duplicate records	check before insert + global handler for database IntegrityError	auth.register, check_duplicate_title, create_review, main.integrity_error_handler	Email is already registered
Business-rule failures	if-checks inside endpoints	book_session, cancel_session, create_review, delete_skill, admin	Cannot cancel a cancelled session

- All errors use `raise HTTPException(status_code, detail)`. FastAPI turns it into `{"detail": "..."}`.
- Duplicates are checked twice: in our code first, and by PostgreSQL unique constraints as a safety net. If two requests race, the database error is caught and still becomes a clean 409.

13.3 The code

```
app/routers/skills.py — missing resource and duplicate

def get_skill_or_404(skill_id: int, db: Session) -> Skill:
    skill = db.get(Skill, skill_id)
    if skill is None:
        raise HTTPException(status.HTTP_404_NOT_FOUND, "Skill not found")
    return skill


def check_duplicate_title(db: Session, mentor_id: int, title: str, exclude_id: int | None = None):
    stmt = select(Skill).where(Skill.mentor_id == mentor_id, Skill.title.ilike(title))
    if exclude_id:
        stmt = stmt.where(Skill.id != exclude_id)
    if db.scalar(stmt):
        raise HTTPException(status.HTTP_409_CONFLICT, "You already have a skill with this title")
```

```
app/main.py — global safety net

@app.exception_handler(IntegrityError)
async def integrity_error_handler(request: Request, exc: IntegrityError):
    """Safety net for duplicate records that slip past the explicit checks."""
    return JSONResponse(status_code=status.HTTP_409_CONFLICT, content={"detail": "Duplicate or conflicting
    record"})
```

Code	What it means
`@app.exception_handler(IntegrityError)`	Registers a handler for one kind of exception for the whole app.

Code	What it means
`IntegrityError`	Raised by SQLAlchemy when PostgreSQL rejects a row (e.g. a duplicate unique value).
`JSONResponse(status_code=409, ...)`	Instead of a 500 crash, the client gets a clean 409.

```
app/routers/sessions.py — business rules while booking

@router.post("", response_model=SessionOut, status_code=status.HTTP_201_CREATED)
def book_session(
    data: SessionCreate,
    user: User = Depends(require_roles(Role.STUDENT)),
    db: Session = Depends(get_db),
):
    skill = db.get(Skill, data.skill_id)
    if skill is None:
        raise HTTPException(status.HTTP_404_NOT_FOUND, "Skill not found")
    if not skill.mentor.is_active:
        raise HTTPException(status.HTTP_400_BAD_REQUEST, "This mentor is not available")

    # Business rule: the mentor cannot have two booked sessions that overlap
    new_start = data.scheduled_at
    new_end = new_start + timedelta(minutes=data.duration_minutes)
    booked = db.scalars(
        select(MentoringSession).where(
            MentoringSession.mentor_id == skill.mentor_id,
            MentoringSession.status == SessionStatus.BOOKED,
        )
    ).all()
    for s in booked:
        start = as_utc(s.scheduled_at)
        end = start + timedelta(minutes=s.duration_minutes)
        if start < new_end and new_start < end:
            raise HTTPException(status.HTTP_409_CONFLICT, "Mentor already has a session booked at this time")

    session = MentoringSession(
        student_id=user.id,
        mentor_id=skill.mentor_id,
        skill_id=skill.id,
        scheduled_at=data.scheduled_at,
        duration_minutes=data.duration_minutes,
        notes=data.notes,
    )
    db.add(session)
    db.commit()
    db.refresh(session)
    return session
```

Code	What it means
`if skill is None: ... 404`	Missing resource.
`if not skill.mentor.is_active: ... 400`	Business rule: blocked mentors cannot be booked.
`if start < new_end and new_start < end: ... 409`	Business rule: no overlapping sessions for one mentor.

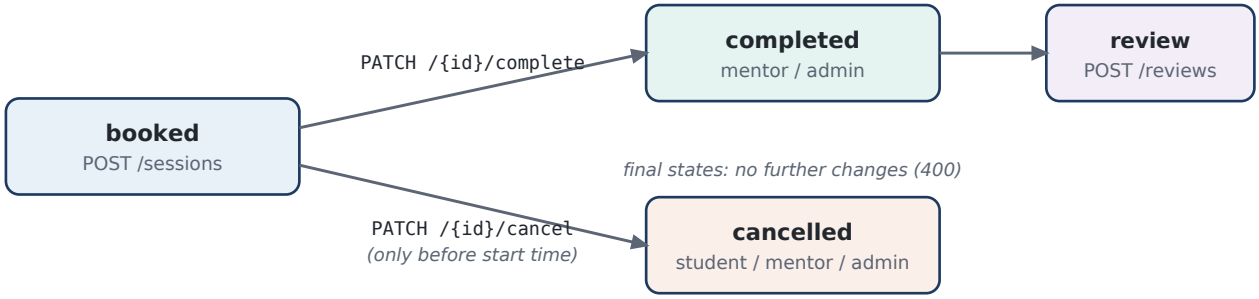


Figure. Session status rules enforced by cancel, complete and create_review.

13.4 Every error message our API can return

Collected automatically from the source code:

Code	Message	Raised in
400	You cannot change your own role	admin.py · change_role()
400	You cannot deactivate yourself	admin.py · change_status()
400	You cannot delete yourself	admin.py · delete_user()
400	You can only review completed sessions	reviews.py · create_review()
400	This mentor is not available	sessions.py · book_session()
400	Cannot cancel a {session.status} session	sessions.py · cancel_session()
400	Cannot cancel a session that has already started	sessions.py · cancel_session()
400	Cannot complete a {session.status} session	sessions.py · complete_session()
400	Cancel upcoming sessions for this skill first	skills.py · delete_skill()
401	Invalid email or password	auth.py · login()
401	Invalid or missing token	security.py
403	Account is deactivated	auth.py · login()
403	You can only review your own sessions	reviews.py · create_review()
403	You can only delete your own reviews	reviews.py · delete_review()
403	You can only edit your own reviews	reviews.py · update_review()
403	You do not have permission to perform this action	security.py
403	This is not your session	sessions.py · get_session_for_user()
403	You can only modify your own skills	skills.py · check_owner()
404	Session not found	admin.py · delete_session()
404	User not found	admin.py · get_user_or_404()
404	Review not found	reviews.py · get_review_or_404()
404	Skill not found	sessions.py · book_session()
404	Mentor not found	users.py · get_mentor_profile()
409	Email is already registered	auth.py · register()
409	This session has already been reviewed	reviews.py · create_review()
409	Mentor already has a session booked at this time	sessions.py · book_session()
409	You already have a skill with this title	skills.py · check_duplicate_title()
422	(Pydantic messages, e.g. “Field required”)	schemas.py, parameters
409	Duplicate or conflicting record	main.py · integrity_error_handler()

13.5 Input → output: one example of each case

Invalid input

REQUEST POST /auth/register (no token)

```
{
  "email": "x@skillswap.com",
  "password": "Abcd1234"
}
```

RESPONSE 422 Unprocessable Content

```
{
  "detail": [
    {
      "type": "missing",
      "loc": [
        "body",
        "name"
      ],
      "msg": "Field required"
    }
  ]
}
```

Missing resource

REQUEST POST /sessions (Authorization: Bearer <token>)

```
{
  "skill_id": 999999,
  "scheduled_at": "2026-10-07T09:00:00Z",
  "duration_minutes": 60
}
```

RESPONSE 404 Not Found

```
{
  "detail": "Skill not found"
}
```

Duplicate record

REQUEST POST /skills (Authorization: Bearer <token>)

```
{
  "title": "HTML & CSS Basics",
  "category": "Web Development",
  "level": "beginner"
}
```

RESPONSE 409 Conflict

```
{
  "detail": "You already have a skill with this title"
}
```

Business rule: double booking

REQUEST POST /sessions (Authorization: Bearer <token>)

```
{
  "skill_id": 7,
  "scheduled_at": "2026-10-06T09:30:00Z",
  "duration_minutes": 30
}
```

RESPONSE 409 Conflict

```
{
  "detail": "Mentor already has a session booked at this time"
}
```

Business rule: only completed sessions can be reviewed

REQUEST `POST /reviews` (Authorization: Bearer <token>)

```
{
  "session_id": 8,
  "rating": 4
}
```

RESPONSE 400 Bad Request

```
{
  "detail": "You can only review completed sessions"
}
```

Business rule: an admin cannot deactivate themselves

REQUEST `PATCH /admin/users/1/status` (Authorization: Bearer <token>)

```
{
  "is_active": false
}
```

RESPONSE 400 Bad Request

```
{
  "detail": "You cannot deactivate yourself"
}
```

Say this when presenting

- “Every problem becomes a clean JSON error with the right code — never a crash.”
- Go through the four required cases and show one real example each.
- Show the IntegrityError handler: “the database is our second line of defence against duplicates.”

REQUIREMENT 14

Environment Variables

Group 5
Member 5

> **Expected implementation:** *Keep database credentials and JWT secrets in environment variables/.env.*

14.1 What it means

Passwords and secret keys must not be written in the source code, because code is shared and uploaded. They are kept in **environment variables**, usually loaded from a `.env` file that is never committed.

14.2 How we used it in SkillSwap

- `.env` (private) holds the real values; `.env.example` (shared) is a template with dummy values.
- `.gitignore` contains `.env`, so it is never uploaded.
- `app/config.py` loads the values with **pydantic-settings** into one `settings` object. Real environment variables override the file.
- `DATABASE_URL` and `JWT_SECRET_KEY` have no default: the server **refuses to start** if they are missing.

Variable	Used for	Used in
DATABASE_URL	PostgreSQL / Supabase connection incl. username and password	database.py · create_engine
JWT_SECRET_KEY	signing and checking tokens	security.py · create_access_token, get_current_user
JWT_ALGORITHM	HS256	security.py
ACCESS_TOKEN_EXPIRE_MINUTES	token lifetime (60)	security.py · create_access_token
ADMIN_EMAIL, ADMIN_PASSWORD	the default admin account	main.py · create_default_admin

14.3 The code

```
app/config.py

from pydantic_settings import BaseSettings, SettingsConfigDict


class Settings(BaseSettings):
    """All secrets/config are read from environment variables or the .env file."""

    database_url: str
    jwt_secret_key: str
    jwt_algorithm: str = "HS256"
    access_token_expire_minutes: int = 60
    admin_email: str = "admin@skillswap.com"
    admin_password: str = "Admin@123"

    model_config = SettingsConfigDict(env_file=".env")


settings = Settings()
```

Code	What it means
`class Settings(BaseSettings)`	pydantic-settings reads each field from an environment variable with the same name (case-insensitive).
`database_url: str`	Required, no default → startup error if missing.

Code	What it means
`access_token_expire_minutes: int = 60`	Optional with a default; converted to int automatically.
`SettingsConfigDict(env_file=".env")`	Also read values from the .env file.
`settings = Settings()`	Loaded once; every module imports this object.

```
.env.example (the template we share)

# PostgreSQL connection (local or Supabase)
# Supabase example: postgresql+psycopg://postgres:<password>@db.<project>.supabase.co:5432/postgres
DATABASE_URL=postgresql+psycopg://postgres:postgres@localhost:5432/skillswap

# JWT settings
JWT_SECRET_KEY=change-me-to-a-long-random-string
JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60

# Default admin account created on startup
ADMIN_EMAIL=admin@skillswap.com
ADMIN_PASSWORD=Admin@123
```

```
.gitignore

.env
.venv/
__pycache__/
*.db
```

```
how the code reads them

from app.config import settings
engine = create_engine(settings.database_url, pool_pre_ping=True)                    # database.py
jwt.encode(payload, settings.jwt_secret_key, algorithm=settings.jwt_algorithm)      # security.py
```

14.4 Proof

- No password or secret appears in any `.py` file — only `settings.xxx`.
- Starting the server without `DATABASE_URL` stops with a clear “Field required” error from pydantic-settings.
- When we moved to the psycopg 3 driver we changed only `.env` and requirements.txt — no Python code.

Say this when presenting

- “Our database password and JWT secret live only in .env, which is git-ignored.”
- Show config.py and .env.example, then where settings is used.
- “If a required secret is missing, the app refuses to start.”

REQUIREMENT 15

Swagger / OpenAPI

Group 5
Member 5

> **Expected implementation:** *Ensure APIs are documented and testable through FastAPI Swagger UI.*

15.1 What it means

OpenAPI is a standard description of an API (endpoints, inputs, outputs). FastAPI builds it from our code and shows it as **Swagger UI**, a web page where every endpoint can be read and tried.

15.2 How we used it in SkillSwap

URL	What it shows
http://127.0.0.1:8000/docs	Swagger UI — interactive docs, “Try it out”, Authorize button
http://127.0.0.1:8000/redoc	ReDoc — read-only documentation
http://127.0.0.1:8000/openapi.json	the raw OpenAPI 3.1 document (we used it to build tables in this PDF)

What we did in code	What it adds to Swagger
FastAPI(title, description, version)	the page title and description
APIRouter(tags=[...])	endpoints grouped by topic
response_model=...	the exact response shape of every endpoint
Pydantic Field / Query rules and descriptions	required fields, types, min/max, enums, help text
status_code=...	the documented success code
HTTPBearer security	the Authorize button and a lock on all 20 protected routes

```
lines that feed the documentation

app = FastAPI(title="SkillSwap API", description="Student skill sharing platform: ...", version="1.0.0")
router = APIRouter(prefix="/sessions", tags=["Mentoring Sessions"])
duration_minutes: int = Field(ge=30, le=180, description="30 to 180 minutes")
q: str | None = Query(None, min_length=1, description="Search in title and description")
bearer_scheme = HTTPBearer(auto_error=False)
```

15.3 Our Swagger UI

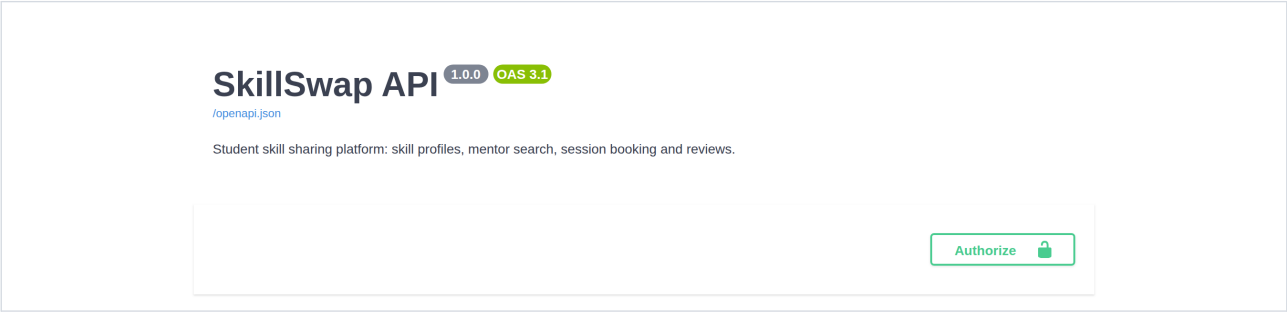


Figure 1. Title, description, OpenAPI link and the Authorize button.

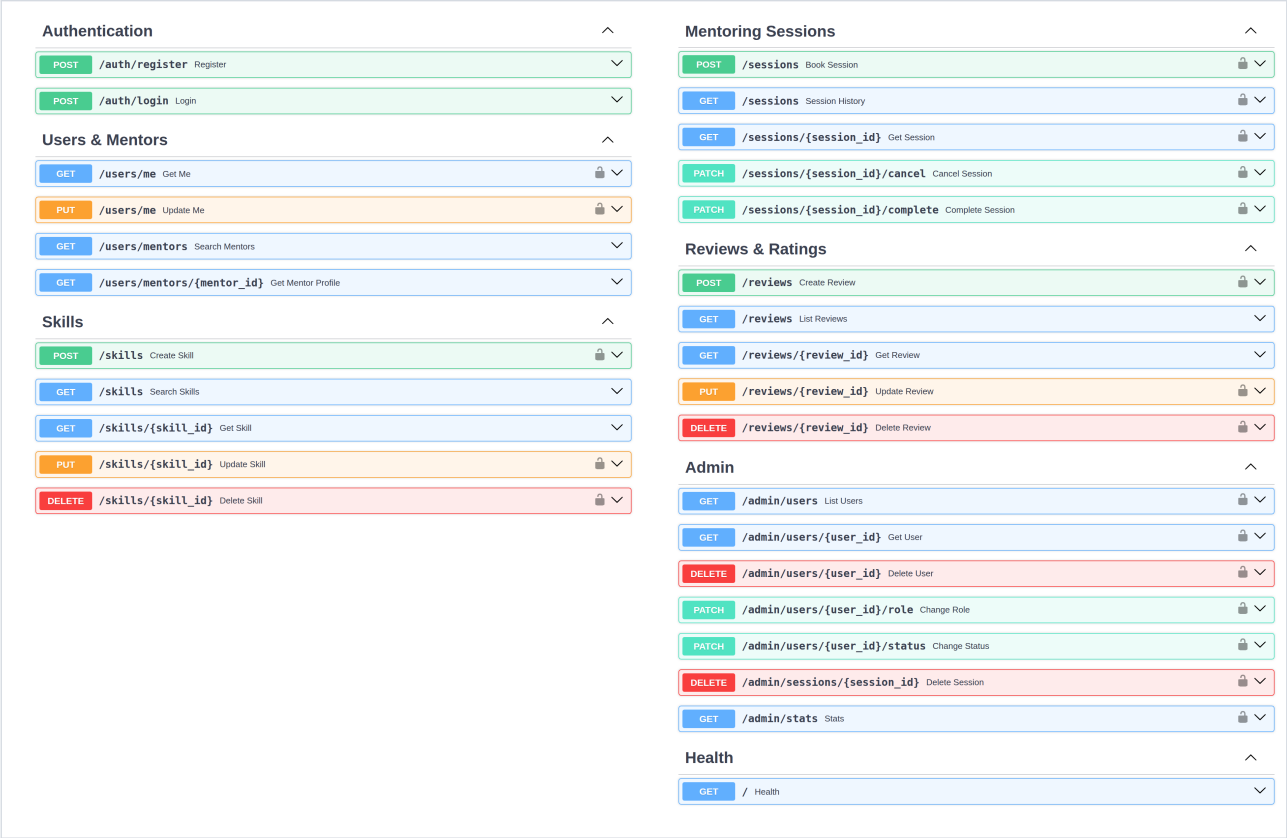


Figure 2. All 29 endpoints grouped by tag; locks mark protected routes.

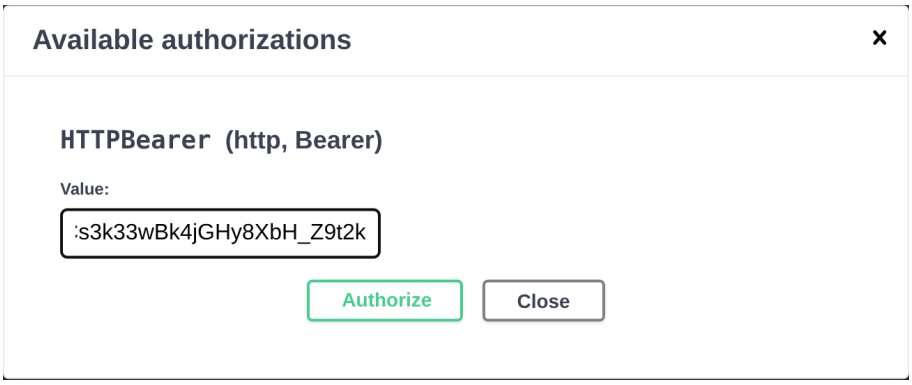


Figure 3. The Authorize dialog: paste the JWT once, Swagger sends it with every request.

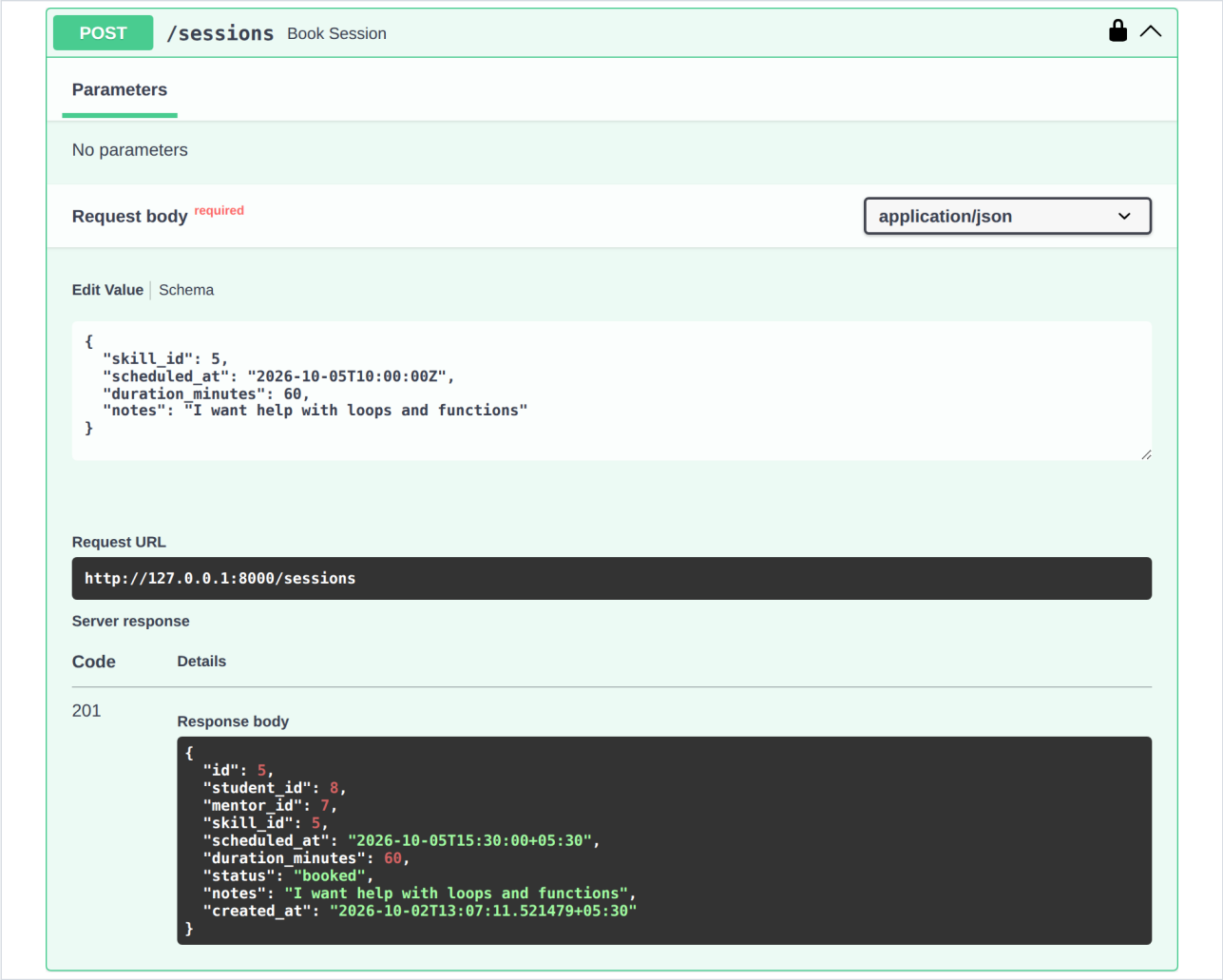


Figure 4. Testing POST /sessions in Swagger: request body, URL and the real 201 response.

How to test in Swagger: open an endpoint → **Try it out** → fill the body or parameters → **Execute** → read the code and response body.

Say this when presenting

- “FastAPI generates our documentation from the code — every endpoint, input and output is described.”
- Open /docs live: show the groups, a schema with its rules, and the lock icons.
- Log in, Authorize, and run one protected request.

REQUIREMENT 16

Postman Testing

Group 5
Member 5

> **Expected implementation:** *Create and test the API using Postman; submit a Postman collection.*

16.1 What it means

Postman is an app for sending API requests and testing the answers. A **collection** is a saved, shareable set of requests that can run all at once.

16.2 How we used it in SkillSwap

- File to submit: `postman/SkillSwap.postman_collection.json` (Postman collection format v2.1).
- **53 requests in 7 folders**, in the order of a real user journey, covering every endpoint group and every error code.
- Each request has a **test script** that checks the exact status code; login requests save the tokens and create requests save the new ids into **collection variables**.
- A **pre-request script** adds a timestamp to emails and calculates future booking times, so the collection can run any number of times.
- Variable `{{baseUrl}}` = `http://127.0.0.1:8000`, so the server address can be changed in one place.

16.3 The scripts

```
Postman JavaScript

// Pre-request script of "Register mentor": unique emails on every run
pm.collectionVariables.set("run", Date.now());

// Tests tab of "Register mentor": check the code and save the new id
pm.test("Status is 201", () => pm.response.to.have.status(201));
pm.collectionVariables.set("mentor_id", pm.response.json().id);

// Tests tab of "Login student": save the token for later requests
pm.test("Status is 200", () => pm.response.to.have.status(200));
pm.collectionVariables.set("student_token", pm.response.json().access_token);

// Pre-request script of "Book session": always a time 2 days in the future
const d = new Date(Date.now() + 2*86400000); d.setUTCMinutes(0,0,0);
pm.collectionVariables.set("slot1", d.toISOString());
```

```
a request that uses saved variables

Authorization tab:  Bearer Token  {{student_token}}
Body (raw JSON):    {"skill_id": "{{skill_id}}", "scheduled_at": "{{slot1}}", "duration_minutes": 60}
```

16.4 All 53 requests and the status code each one expects

#	Folder	Request	Method	Expects
1	Auth	Health check	GET	200
2	Auth	Register mentor	POST	201
3	Auth	Register student	POST	201
4	Auth	Register duplicate email (409)	POST	409
5	Auth	Register invalid data (422)	POST	422
6	Auth	Login admin	POST	200
7	Auth	Login mentor	POST	200
8	Auth	Login student	POST	200
9	Auth	Login wrong password (401)	POST	401
10	Users & Mentors	Get my profile	GET	200
11	Users & Mentors	Get my profile without token (401)	GET	401
12	Users & Mentors	Update my profile	PUT	200
13	Skills	Create skill (mentor)	POST	201
14	Skills	Create second skill (mentor)	POST	201
15	Skills	Create duplicate skill (409)	POST	409
16	Skills	Create skill as student (403)	POST	403
17	Skills	Create skill invalid level (422)	POST	422
18	Skills	Search skills	GET	200
19	Skills	Get skill by id	GET	200
20	Skills	Get missing skill (404)	GET	404
21	Skills	Update skill (owner)	PUT	200
22	Skills	Search mentors by skill	GET	200
23	Skills	Get mentor profile	GET	200
24	Sessions	Book session (student)	POST	201
25	Sessions	Book overlapping slot (409)	POST	409
26	Sessions	Book in the past (422)	POST	422
27	Sessions	Book as mentor (403)	POST	403
28	Sessions	Book second session	POST	201
29	Sessions	Session history (student)	GET	200
30	Sessions	Session history (mentor)	GET	200
31	Sessions	Get session by id	GET	200
32	Sessions	Cancel session	PATCH	200
33	Sessions	Cancel already-cancelled session (400)	PATCH	400
34	Sessions	Complete session (mentor)	PATCH	200
35	Sessions	Complete session as student (403)	PATCH	403
36	Reviews	Review cancelled session (400)	POST	400
37	Reviews	Create review	POST	201
38	Reviews	Duplicate review (409)	POST	409
39	Reviews	Invalid rating (422)	POST	422
40	Reviews	List reviews for mentor	GET	200

#	Folder	Request	Method	Expects
41	Reviews	Update my review	PUT	200
42	Reviews	Search mentors by rating	GET	200
43	Admin	List users (admin)	GET	200
44	Admin	List users as student (403)	GET	403
45	Admin	Platform stats	GET	200
46	Admin	Deactivate student	PATCH	200
47	Admin	Deactivated user cannot login (403)	POST	403
48	Admin	Reactivate student	PATCH	200
49	Admin	Change user role	PATCH	200
50	Admin	Get missing user (404)	GET	404
51	Cleanup (Delete)	Delete review (admin)	DELETE	204
52	Cleanup (Delete)	Delete skill (owner)	DELETE	204
53	Cleanup (Delete)	Delete user (admin)	DELETE	204

16.5 Result

```
run against our PostgreSQL database

$ npx newman run postman/SkillSwap.postman_collection.json

┌─────────────────────────┬─────────────────────┬─────────────────────┐
│                         │            executed │              failed │
├─────────────────────────┼─────────────────────┼─────────────────────┤
│              iterations │                   1 │                   0 │
│                requests │                  53 │                   0 │
│            test-scripts │                  53 │                   0 │
│      prerequest-scripts │                   2 │                   0 │
│              assertions │                  53 │                   0 │
└─────────────────────────┴─────────────────────┴─────────────────────┘
```

In Postman: **Import** the file → right-click **SkillSwap API** → **Run collection** → **Run**.

Say this when presenting

- “Our Postman collection has 53 requests that test every endpoint and every error code.”
- Explain the scripts: tokens and ids are saved automatically, so the requests chain like a real user.
- Run it live: 53 of 53 passed. “This is the collection we submit.”

Summary: all 16 requirements

#	Requirement	Main evidence	Group
1	FastAPI REST API	6 routers, 29 REST endpoints, main.py	1
2	CRUD APIs	CRUD for users, skills, sessions, reviews; 201 / 200 / 204	1
3	PostgreSQL	skillswap database, 4 tables with keys and constraints	1
4	Database integration	SQLAlchemy ORM, get_db, generated SQL	2
5	Pydantic validation	10 request schemas; required, types, ranges, formats, rules → 422	2
6	JWT authentication	register, login, HS256 token with sub / role / exp	2
7	Password hashing	bcrypt cost 12 with salt; only hashes in the database	3
8	Authorization	ownership checks, filtered history, ids from token	3
9	RBAC	student, mentor, admin + require_roles	3
10	Protected routes	get_current_user dependency; 20 protected endpoints	4
11	Path & query parameters	5 path ids; search, filters, dates, rating, paging	4
12	HTTP status codes	200, 201, 204, 400, 401, 403, 404, 409, 422	4
13	Exception handling	HTTPException, 404 helpers, IntegrityError → 409, rule checks	5
14	Environment variables	.env + pydantic-settings, git-ignored	5
15	Swagger / OpenAPI	/docs, tags, schemas, Authorize	5
16	Postman testing	53 requests, 53 passed	5

How to run the project

```
terminal

cd SDPP
source .venv/bin/activate
uvicorn app.main:app --reload          # http://127.0.0.1:8000/docs
# admin login: admin@skillswap.com / Admin@123
```